

CHESS TRAINER - Marco Teórico de Machine Learning

**Versión:** 1.0  
**Fecha:** 4 de Febrero de 2026  
**Objetivo:** Documentar fundamentos teóricos de algoritmos ML aplicados a chess\_trainer

ÍNDICE

1. [Introducción](#)
2. [Regresión Lineal Simple y Múltiple](#)
3. [Regresión Logística](#)
4. [K-Nearest Neighbors \(KNN\)](#)
5. [K-Means Clustering](#)
6. [Naive Bayes](#)
7. [Random Forest](#)
8. [Gradient Boosting](#)
9. [Support Vector Machines \(SVM\)](#)
10. [Neural Networks \(MLP\)](#)
11. [Recurrent Networks \(LSTM/GRU\)](#)
12. [Sobreajuste y Subajuste](#)
13. [Guía de Selección de Algoritmos](#)
14. [Referencias](#)

INTRODUCCIÓN

Este documento proporciona el marco teórico de los algoritmos de Machine Learning utilizados y planificados en **chess\_trainer**. Cada algoritmo incluye:

- **Teoría conceptual:** Fundamentos matemáticos y estadísticos
- **Fórmulas clave:** Representación matemática
- **Aplicación en Chess Trainer:** Uso específico en nuestro contexto
- **Ejemplo concreto:** Código Python aplicado al ajedrez
- **Ventajas y desventajas:** Cuándo usar cada algoritmo
- **Casos de uso:** Problemas específicos que resuelve

Contexto del Proyecto

**Chess Trainer** es un sistema de análisis y entrenamiento ajedrecístico que utiliza ML para:

- Predecir **error\_label** de jugadas (blunder, mistake, inaccuracy, good, brilliant)
- Identificar **patrones tácticos** comunes
- Generar **recomendaciones personalizadas** de mejora
- Analizar **secuencias temporales** de errores
- Crear **perfiles de jugador** por estilo

1. REGRESIÓN LINEAL SIMPLE Y MÚLTIPLE

 Teoría

La **regresión lineal** modela la relación entre una variable dependiente  $y$  y una o más variables independientes  $X$  mediante una ecuación lineal.

**Regresión Lineal Simple:**  $y = \beta_0 + \beta_1 x + \epsilon$

**Regresión Lineal Múltiple:**  $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$

Donde:

- $y$ : Variable objetivo (target)
- $x_i$ : Features (variables independientes)
- $\beta_i$ : Coeficientes (pesos)
- $\epsilon$ : Error residual

**Método de optimización:** Mínimos Cuadrados Ordinarios (OLS)  $\min_{\beta} \sum_{i=1}^n (y_i - \hat{y}_i)^2$

## Aplicación en Chess Trainer

**Predicción de valores continuos:**

1. **Predecir score\_diff** (diferencia de evaluación en centipawns)
2. **Estimar pérdida de ventaja** en una posición
3. **Analizar correlación** entre features y rendimiento
4. **Predicción de tiempo de jugada** basado en complejidad

## Ejemplo Concreto

```
from sklearn.linear_model import LinearRegression
import pandas as pd
import numpy as np

# Cargar datos
df = pd.read_parquet('data/export/personal/features.parquet')

# Features para predecir score_diff
features = [
    'material_balance',
    'branching_factor',
    'self_mobility',
    'opponent_mobility',
    'is_center_controlled'
]

X = df[features]
y = df['score_diff']

# Entrenar modelo
model = LinearRegression()
model.fit(X, y)

# Interpretar coeficientes
for feature, coef in zip(features, model.coef_):
    print(f"{feature}: {coef:.4f}")

# Ejemplo de predicción
new_position = {
    'material_balance': 2,
```

```
'branching_factor': 35,
'self_mobility': 28,
'opponent_mobility': 22,
'is_center_controlled': 1
}
predicted_score = model.predict([list(new_position.values())])[0]
print(f"Score diferencial predicho: {predicted_score:.2f} centipawns")
```

Output esperado:

```
material_balance: 125.34
branching_factor: 2.15
self_mobility: 3.45
opponent_mobility: -2.98
is_center_controlled: 45.67
Score diferencial predicho: 182.45 centipawns
```

- ☒ Ventajas
- **Interpretabilidad:** Coeficientes muestran importancia de cada feature
  - **Rápido:** Entrenamiento y predicción muy eficientes
  - **Bajo riesgo de sobreajuste:** Con pocas features
  - **Baseline excelente:** Punto de partida para modelos complejos

- ☐ Desventajas
- **Solo relaciones lineales:** No capta interacciones complejas
  - **Sensible a outliers:** Valores extremos afectan el modelo
  - **Asume independencia:** Features no deben estar correlacionadas
  - **Limitado para clasificación:** No es su propósito principal

 Casos de Uso en Chess Trainer

| Caso de Uso             | Problema                             | Features                  | Target                |
|-------------------------|--------------------------------------|---------------------------|-----------------------|
| Análisis de evaluación  | ¿Qué features impactan más el score? | Posicionales + tácticos   | score_diff            |
| Predicción de tiempo    | ¿Cuánto tardará en esta posición?    | Complejidad + experiencia | move_time             |
| Correlación de features | ¿Qué features están relacionadas?    | Múltiples                 | Análisis exploratorio |

## 2. REGRESIÓN LOGÍSTICA

 Teoría

La **regresión logística** es un algoritmo de clasificación que modela la probabilidad de pertenencia a una clase usando la función logística (sigmoide).

**Función Sigmoide:**  $\sigma(z) = \frac{1}{1 + e^{-z}}$

**Modelo Logístico (binario):**  $P(y=1|X) = \sigma(\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n)$

**Regresión Logística Multiclase (Softmax):**  $P(y=k|X) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$

**Función de pérdida:** Log-Loss (Cross-Entropy) 
$$L = -\sum_{i=1}^n [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$$

## 🔗 Aplicación en Chess Trainer

**Clasificación de error\_label** (Fase 1 del roadmap):

1. **Baseline principal** para clasificación multiclase
2. **Predicción de probabilidades** de cada tipo de error
3. **Feature selection** con L1 regularization
4. **Interpretabilidad** de decisiones del modelo

## 📖 Ejemplo Concreto

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix
import pandas as pd

# Cargar dataset etiquetado
df = pd.read_sql_query("""
    SELECT
        score_diff, material_balance, material_total,
        branching_factor, self_mobility, opponent_mobility,
        move_number, is_center_controlled, threatens_mate,
        error_label
    FROM move_features
    WHERE error_label IS NOT NULL
""", engine)

# Preparar datos
feature_cols = [
    'score_diff', 'material_balance', 'branching_factor',
    'self_mobility', 'opponent_mobility', 'move_number',
    'is_center_controlled', 'threatens_mate'
]

X = df[feature_cols]
y = df['error_label']

# Split y escalado
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Modelo con regularización L2 (Ridge)
model_l2 = LogisticRegression(
    penalty='l2',
    C=1.0, # Inverso de la fuerza de regularización
    multi_class='multinomial',
    solver='lbfgs',
    max_iter=1000,
    random_state=42
)
```

```

)

model_l2.fit(X_train_scaled, y_train)

# Evaluación
y_pred = model_l2.predict(X_test_scaled)
y_proba = model_l2.predict_proba(X_test_scaled)

print("=== CLASSIFICATION REPORT ===")
print(classification_report(y_test, y_pred))

print("\n=== CONFUSION MATRIX ===")
print(confusion_matrix(y_test, y_pred))

# Predicción con probabilidades
sample_move = X_test.iloc[0]
proba = model_l2.predict_proba([scaler.transform([sample_move])])[0]
classes = model_l2.classes_

print("\n=== PREDICCIÓN EJEMPLO ===")
for cls, prob in zip(classes, proba):
    print(f"{cls}: {prob*100:.2f}%")

```

### Output esperado:

```

=== CLASSIFICATION REPORT ===

```

|            | precision | recall | f1-score | support |
|------------|-----------|--------|----------|---------|
| blunder    | 0.82      | 0.78   | 0.80     | 150     |
| good       | 0.88      | 0.92   | 0.90     | 300     |
| inaccuracy | 0.75      | 0.71   | 0.73     | 140     |
| mistake    | 0.79      | 0.82   | 0.80     | 160     |
| accuracy   |           |        | 0.83     | 750     |
| macro avg  | 0.81      | 0.81   | 0.81     | 750     |

```

=== CONFUSION MATRIX ===
[[117  8 15 10]  # blunder
 [  5 276 12  7]  # good
 [ 18 10 99 13]  # inaccuracy
 [ 12  4 13 131]] # mistake

=== PREDICCIÓN EJEMPLO ===
blunder: 8.45%
good: 65.32%
inaccuracy: 18.23%
mistake: 8.00%

```

### 🔍 Feature Selection con L1 (Lasso)

```

# Modelo con regularización L1 para feature selection
model_l1 = LogisticRegression(
    penalty='l1',
    C=0.5,
    multi_class='multinomial',

```

```
solver='saga', # Soporta L1
max_iter=1000,
random_state=42
)

model_l1.fit(X_train_scaled, y_train)

# Features importantes (coeficientes != 0)
feature_importance = pd.DataFrame({
    'feature': feature_cols,
    'abs_coef': np.abs(model_l1.coef_).mean(axis=0)
}).sort_values('abs_coef', ascending=False)

print("\n=== FEATURE IMPORTANCE (L1) ===")
print(feature_importance)
```

- ✓ Ventajas
- **Probabilidades calibradas:** Output como probabilidades [0,1]
  - **Interpretable:** Coeficientes muestran impacto de features
  - **Regularización built-in:** L1/L2 previene overfitting
  - **Rápido y escalable:** Eficiente con datasets grandes
  - **Baseline estándar:** Punto de referencia para otros modelos

- ✗ Desventajas
- **Solo relaciones lineales:** No capta interacciones complejas
  - **Asume clases separables:** Limitado con clases mezcladas
  - **Sensible a desbalanceo:** Requiere ajuste de class\_weight
  - **Feature engineering crucial:** Necesita features bien diseñadas

🔗 Casos de Uso en Chess Trainer

| Caso de Uso       | Problema                  | Configuración                   | Métricas          |
|-------------------|---------------------------|---------------------------------|-------------------|
| Baseline Fase 1   | Clasificación error_label | L2, C=1.0                       | F1 macro > 0.70   |
| Feature selection | ¿Qué features importan?   | L1, C=0.5                       | Coeficientes != 0 |
| Blunder detection | Detectar errores graves   | Binary, class_weight='balanced' | Recall blunders   |
| Quick predictions | Predicción en tiempo real | Modelo pre-entrenado            | Latencia < 10ms   |

### 3. K-NEAREST NEIGHBORS (KNN)

📊 Teoría

**K-Nearest Neighbors (KNN)** es un algoritmo de clasificación/regresión basado en instancias que predice el valor de un dato nuevo basándose en los K vecinos más cercanos.

**Algoritmo:**

1. Calcular distancia entre punto nuevo y todos los puntos de training
2. Seleccionar los K vecinos más cercanos
3. **Clasificación:** Votar por la clase más frecuente
4. **Regresión:** Promediar los valores de los vecinos

**Distancias comunes:**

**Euclidiana:**  $d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$

**Manhattan:**  $d(x, y) = \sum_{i=1}^n |x_i - y_i|$

**Minkowski (generalización):**  $d(x, y) = (\sum_{i=1}^n |x_i - y_i|^p)^{1/p}$

🔄 Aplicación en Chess Trainer

**Búsqueda de similitud y recomendaciones:**

1. **Encontrar jugadas similares** a una posición dada
2. **Recomendar posiciones de entrenamiento** similares a errores del jugador
3. **Clasificación local** cuando patrones son complejos
4. **Baseline para comparar** con modelos más sofisticados

## 📁 Ejemplo Concreto

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
import pandas as pd
import numpy as np

# Cargar features
df = pd.read_parquet('data/export/combined_features.parquet')

# Features para similitud
features = [
    'material_balance', 'branching_factor',
    'self_mobility', 'opponent_mobility',
    'score_diff', 'move_number'
]

X = df[features]
y = df['error_label']

# Escalar features (crucial para KNN)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Entrenar KNN
knn = KNeighborsClassifier(
    n_neighbors=5,
    weights='distance', # Vecinos cercanos pesan más
    metric='euclidean',
    algorithm='auto'
)

knn.fit(X_scaled, y)

# Función para encontrar jugadas similares
def find_similar_moves(move_features, n_neighbors=5):
    """
    Encuentra las jugadas más similares a una dada
    """
    move_scaled = scaler.transform([move_features])
```

```

# Encontrar vecinos más cercanos
distances, indices = knn.kneighbors(move_scaled, n_neighbors=n_neighbors)

similar_moves = []
for idx, dist in zip(indices[0], distances[0]):
    similar_move = {
        'game_id': df.iloc[idx]['game_id'],
        'move_number': df.iloc[idx]['move_number'],
        'error_label': df.iloc[idx]['error_label'],
        'distance': dist,
        'features': df.iloc[idx][features].to_dict()
    }
    similar_moves.append(similar_move)

return similar_moves

# Ejemplo: Jugada con material balance negativo
problematic_move = {
    'material_balance': -3,
    'branching_factor': 45,
    'self_mobility': 18,
    'opponent_mobility': 32,
    'score_diff': -250,
    'move_number': 15
}

print("=== JUGADAS SIMILARES A POSICIÓN PROBLEMÁTICA ===")
similar = find_similar_moves(list(problematic_move.values()))

for i, move in enumerate(similar, 1):
    print(f"\n{i}. Distancia: {move['distance']:.4f}")
    print(f"    Error: {move['error_label']}")
    print(f"    Game: {move['game_id']}, Jugada: {move['move_number']}")

# Sistema de recomendaciones
def recommend_training_positions(player_errors, n_recommendations=10):
    """
    Recomienda posiciones de entrenamiento basadas en errores del jugador
    """
    # Obtener features de errores del jugador
    error_features = player_errors[features].values
    error_features_scaled = scaler.transform(error_features)

    # Encontrar posiciones similares en dataset de entrenamiento
    all_recommendations = []

    for error_feat in error_features_scaled:
        distances, indices = knn.kneighbors([error_feat], n_neighbors=3)
        all_recommendations.extend(indices[0])

    # Retornar posiciones únicas más frecuentes
    unique_recs = np.unique(all_recommendations, return_counts=True)
    top_indices = unique_recs[0][np.argsort(unique_recs[1])[-n_recommendations:]]

    return df.iloc[top_indices]

# Ejemplo de uso
player_blunders = df[df['error_label'] == 'blunder'].sample(5)
recommendations = recommend_training_positions(player_blunders)

```



```
print("\n=== POSICIONES RECOMENDADAS PARA ENTRENAMIENTO ===")
print(recommendations[['game_id', 'move_number', 'error_label', 'score_diff']])
```

Output esperado:

```
=== JUGADAS SIMILARES A POSICIÓN PROBLEMÁTICA ===

1. Distancia: 0.2456
   Error: mistake
   Game: abc123, Jugada: 14

2. Distancia: 0.3102
   Error: blunder
   Game: def456, Jugada: 16

3. Distancia: 0.3845
   Error: mistake
   Game: ghi789, Jugada: 13

=== POSICIONES RECOMENDADAS PARA ENTRENAMIENTO ===
game_id  move_number  error_label  score_diff
0  xyz123      15      blunder      -280
1  abc456      12      mistake      -150
2  def789      18      blunder      -320
...
```

- ☒ Ventajas
- **No asume distribución:** Funciona con datos no lineales
  - **Simple e intuitivo:** Fácil de entender y explicar
  - **Flexible:** Sirve para clasificación y regresión
  - **No requiere training:** Lazy learning (training instantáneo)
  - **Actualizable:** Nuevos datos sin reentrenamiento

- ☒ Desventajas
- **Predicción lenta:** Calcula distancia a todos los puntos
  - **Sensible a escala:** Requiere normalización obligatoria
  - **Curse of dimensionality:** Performance degrada con muchos features
  - **Requiere mucha memoria:** Almacena todo el dataset
  - **Sensible a ruido:** Outliers afectan predicciones

🔗 Casos de Uso en Chess Trainer

| Caso de Uso              | Problema                    | K óptimo | Métricas               |
|--------------------------|-----------------------------|----------|------------------------|
| Similar moves            | Buscar jugadas parecidas    | 5-10     | Distancia euclidiana   |
| Training recommendations | Recomendar ejercicios       | 3-5      | Frecuencia de patrones |
| Local classification     | Zonas de decisión complejas | 7-15     | Accuracy local         |
| Baseline comparison      | Comparar con modelos        | 5        | F1 vs otros modelos    |

## 4. K-MEANS CLUSTERING

### Teoría

**K-Means** es un algoritmo de clustering (aprendizaje no supervisado) que agrupa datos en K clusters minimizando la varianza intra-cluster.

#### Algoritmo:

1. Inicializar K centroides aleatoriamente
2. **Asignación:** Asignar cada punto al centroide más cercano
3. **Actualización:** Recalcular centroides como promedio de puntos asignados
4. Repetir 2-3 hasta convergencia

**Función objetivo (minimizar):**  $J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$

Donde:

- $C_k$ : Cluster k
- $\mu_k$ : Centroide del cluster k
- $\|x_i - \mu_k\|$ : Distancia euclidiana

**Método del codo:** Determinar K óptimo graficando inercia vs K

### Aplicación en Chess Trainer

#### Agrupamiento de jugadores y patrones:

1. **Clustering por nivel ELO:** Identificar grupos de jugadores similares
2. **Patrones de error:** Agrupar tipos de errores recurrentes
3. **Segmentación de estilos:** Identificar perfiles de juego
4. **Análisis de aperturas:** Agrupar aperturas por características

### Ejemplo Concreto

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Cargar datos de jugadores
df_players = pd.read_sql_query("""
    SELECT
        player_id,
        standardized_elo,
        AVG(CASE WHEN error_label = 'blunder' THEN 1 ELSE 0 END) as blunder_rate,
        AVG(CASE WHEN error_label = 'mistake' THEN 1 ELSE 0 END) as mistake_rate,
        AVG(CASE WHEN error_label = 'inaccuracy' THEN 1 ELSE 0 END) as inaccuracy_rate,
        AVG(branching_factor) as avg_complexity,
        AVG(move_time) as avg_time
    FROM move_features
    GROUP BY player_id, standardized_elo
    HAVING COUNT(*) > 50 -- Suficientes datos
""", engine)
```

```

# Features para clustering
features = [
    'standardized_elo',
    'blunder_rate',
    'mistake_rate',
    'inaccuracy_rate',
    'avg_complexity',
    'avg_time'
]

X = df_players[features]

# Escalar features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Método del codo para encontrar K óptimo
inertias = []
silhouette_scores = []
K_range = range(2, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    inertias.append(kmeans.inertia_)
    silhouette_scores.append(silhouette_score(X_scaled, kmeans.labels_))

# Graficar método del codo
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(K_range, inertias, 'bo-')
plt.xlabel('Número de Clusters (K)')
plt.ylabel('Inercia')
plt.title('Método del Codo')
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(K_range, silhouette_scores, 'ro-')
plt.xlabel('Número de Clusters (K)')
plt.ylabel('Silhouette Score')
plt.title('Silhouette Score por K')
plt.grid(True)

plt.tight_layout()
plt.savefig('clustering_analysis.png')

# Seleccionar K óptimo (ejemplo: K=5)
optimal_k = 5
kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
df_players['cluster'] = kmeans.fit_predict(X_scaled)

# Analizar clusters
print("=== ANÁLISIS DE CLUSTERS ===\n")
for cluster_id in range(optimal_k):
    cluster_data = df_players[df_players['cluster'] == cluster_id]

    print(f"CLUSTER {cluster_id} (n={len(cluster_data)})")
    print(f"  ELO promedio: {cluster_data['standardized_elo'].mean():.0f}")

```

```

print(f"  Blunder rate: {cluster_data['blunder_rate'].mean()*100:.2f}%")
print(f"  Mistake rate: {cluster_data['mistake_rate'].mean()*100:.2f}%")
print(f"  Inaccuracy rate: {cluster_data['inaccuracy_rate'].mean()*100:.2f}%")
print(f"  Complejidad avg: {cluster_data['avg_complexity'].mean():.1f}")
print()

# Interpretar clusters (etiquetado manual)
cluster_labels = {
    0: "Principiantes - Alto error rate",
    1: "Intermedios - Errores tácticos",
    2: "Avanzados - Errores posicionales",
    3: "Expertos - Errores sutiles",
    4: "Masters - Muy pocos errores"
}

df_players['cluster_label'] = df_players['cluster'].map(cluster_labels)

# Sistema de recomendaciones por cluster
def get_cluster_recommendations(cluster_id):
    """
    Genera recomendaciones específicas para cada cluster
    """
    recommendations = {
        0: [
            "Practicar visión de jugadas (1-2 jugadas adelante)",
            "Estudiar mates básicos",
            "Evitar blunders de material",
            "Ejercicios de táctica básica"
        ],
        1: [
            "Entrenamiento táctico intermedio",
            "Reconocimiento de patrones (pins, forks)",
            "Mejorar cálculo de variantes",
            "Estudiar finales básicos"
        ],
        2: [
            "Profundizar comprensión posicional",
            "Estructuras de peones",
            "Planes de medio juego",
            "Táctica compleja"
        ],
        3: [
            "Preparación de aperturas profunda",
            "Finales técnicos",
            "Optimización de tiempo",
            "Análisis de partidas de GM"
        ],
        4: [
            "Refinamiento de estilo",
            "Preparación específica de oponentes",
            "Optimización psicológica",
            "Innovación en aperturas"
        ]
    }
    return recommendations.get(cluster_id, [])

print("=== RECOMENDACIONES POR CLUSTER ===\n")
for cluster_id, label in cluster_labels.items():
    print(f"{label}:")

```

```
recs = get_cluster_recommendations(cluster_id)
for rec in recs:
    print(f" - {rec}")
print()
```

**Output esperado:**

=== ANÁLISIS DE CLUSTERS ===

CLUSTER 0 (n=245)

ELO promedio: 1150  
Blunder rate: 8.45%  
Mistake rate: 12.30%  
Inaccuracy rate: 18.50%  
Complejidad avg: 28.5

CLUSTER 1 (n=412)

ELO promedio: 1520  
Blunder rate: 4.20%  
Mistake rate: 8.10%  
Inaccuracy rate: 14.20%  
Complejidad avg: 32.8

CLUSTER 2 (n=356)

ELO promedio: 1850  
Blunder rate: 2.10%  
Mistake rate: 5.20%  
Inaccuracy rate: 10.50%  
Complejidad avg: 36.2

CLUSTER 3 (n=198)

ELO promedio: 2150  
Blunder rate: 0.80%  
Mistake rate: 2.50%  
Inaccuracy rate: 6.80%  
Complejidad avg: 40.1

CLUSTER 4 (n=89)

ELO promedio: 2450  
Blunder rate: 0.30%  
Mistake rate: 1.10%  
Inaccuracy rate: 3.50%  
Complejidad avg: 44.5

=== RECOMENDACIONES POR CLUSTER ===

Principiantes - Alto error rate:

- Practicar visión de jugadas (1-2 jugadas adelante)
- Estudiar mates básicos
- Evitar blunders de material
- Ejercicios de táctica básica

...

☒ Ventajas

- **Simple y rápido:** Algoritmo eficiente
- **Escalable:** Funciona con datasets grandes
- **Interpretable:** Centroides tienen significado
- **Flexible:** Se adapta a diferentes tipos de datos

✖ Desventajas

- **Requiere especificar K:** No siempre es obvio
- **Sensible a inicialización:** Múltiples runs necesarios
- **Asume clusters esféricos:** No funciona con formas complejas
- **Sensible a outliers:** Puntos extremos afectan centroides
- **Escala uniforme:** Requiere normalización

🏆 Casos de Uso en Chess Trainer

| Caso de Uso            | Features                  | K óptimo | Objetivo                       |
|------------------------|---------------------------|----------|--------------------------------|
| Segmentación por nivel | ELO + error rates         | 5-7      | Recomendaciones personalizadas |
| Patrones de error      | Features tácticos         | 4-6      | Identificar tipos de errores   |
| Estilos de juego       | Preferencias posicionales | 3-5      | Perfiles de jugador            |
| Aperturas similares    | Características apertura  | 6-10     | Agrupación de repertorio       |

5. NAIVE BAYES

📊 Teoría

**Naive Bayes** es un clasificador probabilístico basado en el teorema de Bayes con la "ingenua" asunción de independencia condicional entre features.

**Teorema de Bayes:**  $P(y|X) = \frac{P(X|y) \cdot P(y)}{P(X)}$

**Naive Bayes Classifier:**  $P(y|x_1, \dots, x_n) = \frac{P(y) \prod_{i=1}^n P(x_i|y)}{P(x_1, \dots, x_n)}$

**Asunción de independencia:**  $P(x_i \mid y, x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_n) = P(x_i \mid y)$

**Variantes:**

- **Gaussian NB:** Features continuos (distribución normal)
- **Multinomial NB:** Conteos/frecuencias (texto)
- **Bernoulli NB:** Features binarios

🏆 Aplicación en Chess Trainer

**Clasificación rápida y probabilística:**

1. **Predicción rápida de aperturas** basado en primeras jugadas
2. **Clasificación de resultado** (win/draw/loss) por features
3. **Detección de patrones tácticos** con features binarios
4. **Baseline muy rápido** para comparación

📄 Ejemplo Concreto

```

from sklearn.naive_bayes import GaussianNB, BernoulliNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score
import pandas as pd
import numpy as np

# Ejemplo 1: Clasificación de error_label con Gaussian NB
df = pd.read_parquet('data/export/combined_features.parquet')

# Features continuos
continuous_features = [
    'score_diff', 'material_balance', 'branching_factor',
    'self_mobility', 'opponent_mobility', 'move_number'
]

X_cont = df[continuous_features]
y = df['error_label']

X_train, X_test, y_train, y_test = train_test_split(
    X_cont, y, test_size=0.2, random_state=42, stratify=y
)

# Gaussian Naive Bayes
gnb = GaussianNB()
gnb.fit(X_train, y_train)
y_pred = gnb.predict(X_test)

print("=== GAUSSIAN NAIVE BAYES ===")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Ejemplo 2: Detección de patrones tácticos con Bernoulli NB
# Features binarios (presencia/ausencia de patrón táctico)
binary_features = [
    'threatens_mate', 'is_forced_move', 'is_center_controlled',
    'is_pawn_endgame', 'is_low_mobility', 'has_castling_rights',
    'discovered_attack', 'pin', 'fork', 'skewer'
]

X_bin = df[binary_features].astype(int)
y_tactical = (df['error_label'].isin(['blunder', 'mistake'])).astype(int)

X_train_bin, X_test_bin, y_train_tac, y_test_tac = train_test_split(
    X_bin, y_tactical, test_size=0.2, random_state=42
)

# Bernoulli Naive Bayes
bnb = BernoulliNB(alpha=1.0) # Laplace smoothing
bnb.fit(X_train_bin, y_train_tac)
y_pred_tac = bnb.predict(X_test_bin)

print("\n=== BERNOULLI NAIVE BAYES (Tactical Error Detection) ===")
print(f"Accuracy: {accuracy_score(y_test_tac, y_pred_tac):.4f}")
print(f"Precision: {precision_score(y_test_tac, y_pred_tac):.4f}")
print(f"Recall: {recall_score(y_test_tac, y_pred_tac):.4f}")

# Ejemplo 3: Predicción de apertura

```

```

def predict_opening_family(first_moves):
    """
    Predice familia de apertura basado en primeras jugadas
    """
    # Simplificación: features basados en primeras 3-4 jugadas
    # En realidad: codificar jugadas como features

    openings_df = pd.read_sql_query("""
        SELECT
            opening_family,
            e4_count, d4_count, c4_count, nf3_count,
            control_center, fianchetto, castle_early
        FROM opening_patterns
    """, engine)

    X_open = openings_df.drop('opening_family', axis=1)
    y_open = openings_df['opening_family']

    mnb = MultinomialNB(alpha=0.5)
    mnb.fit(X_open, y_open)

    # Predecir nueva partida
    new_game_features = encode_moves(first_moves)
    predicted_opening = mnb.predict([new_game_features])[0]
    proba = mnb.predict_proba([new_game_features])[0]

    return predicted_opening, proba.max()

# Uso
opening, confidence = predict_opening_family(['e4', 'e5', 'Nf3', 'Nc6'])
print(f"\n=== PREDICCIÓN DE APERTURA ===")
print(f"Apertura predicha: {opening}")
print(f"Confianza: {confidence*100:.2f}%")

# Probabilidades por clase
def show_error_probabilities(move_features):
    """
    Muestra probabilidades de error para una jugada
    """
    proba = gnb.predict_proba([move_features])[0]
    classes = gnb.classes_

    print("\n=== PROBABILIDADES DE ERROR ===")
    for cls, prob in sorted(zip(classes, proba), key=lambda x: x[1], reverse=True):
        print(f"{cls:15s}: {prob*100:5.2f}%")

# Ejemplo
sample_move = [
    -250, # score_diff (malo)
    -3, # material_balance (desventaja)
    45, # branching_factor (complejo)
    18, # self_mobility (baja)
    35, # opponent_mobility (alta)
    15 # move_number
]
show_error_probabilities(sample_move)

```

**Output esperado:**



```
=== GAUSSIAN NAIVE BAYES ===
Accuracy: 0.7245

Classification Report:
      precision    recall  f1-score   support

 blunder      0.68      0.72      0.70      150
    good      0.78      0.75      0.76      300
 inaccuracy    0.70      0.68      0.69      140
   mistake    0.71      0.74      0.72      160

 accuracy                   0.72      750

=== BERNOULLI NAIVE BAYES (Tactical Error Detection) ===
Accuracy: 0.7856
Precision: 0.7234
Recall: 0.8102

=== PREDICCIÓN DE APERTURA ===
Apertura predicha: Ruy Lopez
Confianza: 78.45%

=== PROBABILIDADES DE ERROR ===
blunder      : 42.30%
mistake      : 28.15%
inaccuracy   : 20.55%
good         :  9.00%
```

- ☑ Ventajas
- **Extremadamente rápido:** Training y predicción muy eficientes
  - **Funciona con pocos datos:** No requiere dataset enorme
  - **Escalable:** Maneja muchos features sin problema
  - **Probabilístico:** Output como probabilidades
  - **Robusto a features irrelevantes:** Asunción de independencia ayuda

- ✗ Desventajas
- **Asunción de independencia:** Raramente cierta en realidad
  - **No capta interacciones:** Features correlacionadas problemáticas
  - **Menor accuracy:** Generalmente peor que modelos complejos
  - **Sensible a distribución:** Gaussian NB asume normalidad

🔗 Casos de Uso en Chess Trainer

| Caso de Uso            | Variante NB | Features               | Objetivo             |
|------------------------|-------------|------------------------|----------------------|
| Quick baseline         | Gaussian    | Continuos              | Comparación rápida   |
| Tactical detection     | Bernoulli   | Binarios (patrones)    | Detección rápida     |
| Opening classification | Multinomial | Frecuencias de jugadas | Identificar apertura |
| Real-time prediction   | Gaussian    | Subset features        | Latencia < 5ms       |

6. RANDOM FOREST

## Teoría

**Random Forest** es un ensemble de árboles de decisión que vota por la predicción más popular. Combina **bagging** (bootstrap aggregating) con **feature randomness**.

### Algoritmo:

1. Crear N árboles de decisión
2. Para cada árbol:
  - Bootstrap sample (muestreo con reemplazo)
  - En cada split: considerar solo  $\sqrt{m}$  features aleatorios
  - Crear árbol sin poda
3. **Predicción clasificación**: Voto mayoritario
4. **Predicción regresión**: Promedio de predicciones

**Feature Importance**: 
$$Importance(f) = \frac{1}{N} \sum_{tree=1}^N \Delta Gini(f, tree)$$

### Out-of-Bag (OOB) Error:

- ~36% de datos no usados en cada árbol (bootstrap)
- Usar OOB samples para validación interna

## Aplicación en Chess Trainer

### Modelo principal para error\_label (Fase 1):

1. **Clasificación multiclase** de error\_label
2. **Feature importance** para entender decisiones
3. **Manejo de no-linealidad** y interacciones
4. **Robusto** a outliers y ruido
5. **Baseline fuerte** difícil de superar

## Ejemplo Concreto

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.metrics import classification_report, confusion_matrix, f1_score
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Cargar dataset
df = pd.read_sql_query("""
    SELECT * FROM move_features
    WHERE error_label IS NOT NULL
""", engine)

# Features
feature_cols = [
    'score_diff', 'material_balance', 'material_total',
    'num_pieces', 'branching_factor', 'self_mobility',
    'opponent_mobility', 'move_number', 'player_color',
    'has_castling_rights', 'is_repetition', 'is_low_mobility',
    'is_center_controlled', 'is_pawn_endgame', 'threatens_mate',
    'is_forced_move', 'discovered_attack', 'pin', 'fork'
```

```
]

X = df[feature_cols]
y = df['error_label']

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Random Forest básico
rf_basic = RandomForestClassifier(
    n_estimators=100,
    max_depth=None,
    min_samples_split=5,
    min_samples_leaf=2,
    max_features='sqrt',
    random_state=42,
    n_jobs=-1,
    oob_score=True # Usar OOB para validación
)

rf_basic.fit(X_train, y_train)

# Evaluación
y_pred = rf_basic.predict(X_test)
y_pred_proba = rf_basic.predict_proba(X_test)

print("=== RANDOM FOREST BASELINE ===")
print(f"OOB Score: {rf_basic.oob_score_: .4f}")
print(f"Test Accuracy: {accuracy_score(y_test, y_pred): .4f}")
print(f"F1 Macro: {f1_score(y_test, y_pred, average='macro'): .4f}")

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred, labels=rf_basic.classes_)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=rf_basic.classes_,
            yticklabels=rf_basic.classes_)
plt.title('Confusion Matrix - Random Forest')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.tight_layout()
plt.savefig('rf_confusion_matrix.png')

# Feature Importance
feature_importance = pd.DataFrame({
    'feature': feature_cols,
    'importance': rf_basic.feature_importances_
}).sort_values('importance', ascending=False)

print("\n=== TOP 10 FEATURES MÁS IMPORTANTES ===")
print(feature_importance.head(10))

# Visualizar feature importance
plt.figure(figsize=(10, 6))
```

```

top_features = feature_importance.head(15)
plt.barh(top_features['feature'], top_features['importance'])
plt.xlabel('Importance')
plt.title('Top 15 Feature Importances')
plt.gca().invert_yaxis()
plt.tight_layout()
plt.savefig('rf_feature_importance.png')

# Hyperparameter tuning con GridSearchCV
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [10, 20, 30, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2']
}

rf_grid = GridSearchCV(
    RandomForestClassifier(random_state=42, n_jobs=-1),
    param_grid,
    cv=5,
    scoring='f1_macro',
    n_jobs=-1,
    verbose=1
)

print("\n=== GRID SEARCH (esto toma tiempo...) ===")
rf_grid.fit(X_train, y_train)

print(f"\nBest Parameters: {rf_grid.best_params_}")
print(f"Best CV F1 Macro: {rf_grid.best_score_:.4f}")

# Modelo optimizado
rf_optimized = rf_grid.best_estimator_
y_pred_opt = rf_optimized.predict(X_test)

print("\n=== RANDOM FOREST OPTIMIZADO ===")
print(f"Test Accuracy: {accuracy_score(y_test, y_pred_opt):.4f}")
print(f"F1 Macro: {f1_score(y_test, y_pred_opt, average='macro'):.4f}")

# Validación cruzada
cv_scores = cross_val_score(
    rf_optimized, X, y,
    cv=5,
    scoring='f1_macro',
    n_jobs=-1
)

print(f"\nCross-Validation F1 Macro: {cv_scores.mean():.4f} ± {cv_scores.std():.4f}")

# Análisis de confusión grave (good ⇌ blunder)
cm_opt = confusion_matrix(y_test, y_pred_opt, labels=rf_optimized.classes_)
class_indices = {cls: i for i, cls in enumerate(rf_optimized.classes_)}

if 'good' in class_indices and 'blunder' in class_indices:
    good_idx = class_indices['good']
    blunder_idx = class_indices['blunder']

    good_as_blunder = cm_opt[good_idx, blunder_idx]

```

```

blunder_as_good = cm_opt[blunder_idx, good_idx]
total_good = cm_opt[good_idx, :].sum()
total_blunder = cm_opt[blunder_idx, :].sum()

confusion_rate = (good_as_blunder + blunder_as_good) / (total_good + total_blunder)

print(f"\n=== ANÁLISIS DE CONFUSIÓN GRAVE ===")
print(f"Good → Blunder: {good_as_blunder} ({good_as_blunder/total_good*100:.2f}%)")
print(f"Blunder → Good: {blunder_as_good} ({blunder_as_good/total_blunder*100:.2f}%)")
print(f"Confusión grave total: {confusion_rate*100:.2f}%)")
print(f"Criterio Fase 1: {'✅ APROBADO' if confusion_rate < 0.05 else '❌ NO CUMPLE'} (< 5%)")

# Función de predicción con explicación
def predict_with_explanation(model, features, feature_names):
    """
    Predice error y muestra features más relevantes
    """
    prediction = model.predict([features])[0]
    proba = model.predict_proba([features])[0]

    # Feature contributions (aproximación simple)
    importance = model.feature_importances_
    feature_values = np.array(features)
    contributions = importance * np.abs(feature_values - np.median(X_train, axis=0))

    contrib_df = pd.DataFrame({
        'feature': feature_names,
        'value': feature_values,
        'contribution': contributions
    }).sort_values('contribution', ascending=False)

    print(f"\n=== PREDICCIÓN ===")
    print(f"Error predicho: {prediction}")
    print(f"Confianza: {proba.max()*100:.2f}%)")
    print(f"\nTop 5 features que influyeron:")
    print(contrib_df.head())

    return prediction, proba

# Ejemplo de uso
sample_move = X_test.iloc[0].values
predict_with_explanation(rf_optimized, sample_move, feature_cols)

```

### Output esperado:

```

=== RANDOM FOREST BASELINE ===
OOB Score: 0.8456
Test Accuracy: 0.8523
F1 Macro: 0.8245

```

#### Classification Report:

|            | precision | recall | f1-score | support |
|------------|-----------|--------|----------|---------|
| blunder    | 0.83      | 0.81   | 0.82     | 150     |
| good       | 0.89      | 0.91   | 0.90     | 300     |
| inaccuracy | 0.78      | 0.76   | 0.77     | 140     |
| mistake    | 0.82      | 0.84   | 0.83     | 160     |

```

accuracy          0.85      750
macro avg         0.83      0.83      0.83      750

=== TOP 10 FEATURES MÁS IMPORTANTES ===
      feature  importance
0      score_diff      0.2845
1  material_balance      0.1623
2  branching_factor      0.1245
3      self_mobility      0.0987
4  opponent_mobility      0.0856
5      move_number      0.0678
6      threatens_mate      0.0543
7  is_center_controlled      0.0432
8      material_total      0.0398
9              pin      0.0354

=== GRID SEARCH ===
Best Parameters: {'max_depth': 30, 'max_features': 'sqrt', 'min_samples_leaf': 2,
'min_samples_split': 5, 'n_estimators': 300}
Best CV F1 Macro: 0.8567

=== RANDOM FOREST OPTIMIZADO ===
Test Accuracy: 0.8678
F1 Macro: 0.8489

Cross-Validation F1 Macro: 0.8512 ± 0.0156

=== ANÁLISIS DE CONFUSIÓN GRAVE ===
Good → Blunder: 8 (2.67%)
Blunder → Good: 5 (3.33%)
Confusión grave total: 2.89%
Criterio Fase 1: ☒ APROBADO (< 5%)

=== PREDICCIÓN ===
Error predicho: mistake
Confianza: 68.45%

Top 5 features que influyeron:
      feature  value  contribution
0      score_diff -145.0      0.3254
1  material_balance   -2.0      0.1876
3      self_mobility   18.0      0.1234
2  branching_factor   45.0      0.0987
6      threatens_mate    0.0      0.0543

```

### ☒ Ventajas

- **Alta accuracy:** Uno de los mejores algoritmos out-of-the-box
- **Robusto:** Maneja outliers, ruido y datos faltantes
- **Feature importance:** Interpretabilidad de decisiones
- **No requiere scaling:** Funciona con diferentes escalas
- **Maneja interacciones:** Capta relaciones no lineales
- **OOB validation:** Validación interna sin CV costoso
- **Paralelizable:** Training rápido con múltiples cores

### ✗ Desventajas

- **Modelo grande:** Consume mucha memoria (N árboles)
- **Predicción lenta:** Más lento que modelos simples
- **Sesgo en features:** Favorece features con más valores
- **Difícil de interpretar:** Cada árbol es complejo
- **Puede sobreajustar:** Con árboles muy profundos

 Casos de Uso en Chess Trainer

| Caso de Uso       | Configuración             | Métricas Objetivo         | Notas            |
|-------------------|---------------------------|---------------------------|------------------|
| Baseline Fase 1   | n=100-300, max_depth=None | F1 > 0.70, Confusión < 5% | Modelo principal |
| Feature selection | n=100, max_depth=10       | Importance > threshold    | Reducir features |
| Quick predictions | n=50, max_depth=15        | Latencia < 50ms           | Modelo ligero    |
| High accuracy     | n=500, optimized          | F1 > 0.85                 | Producción       |

7. GRADIENT BOOSTING (XGBoost, CatBoost)

 Teoría

**Gradient Boosting** es un ensemble secuencial que construye árboles iterativamente, donde cada árbol nuevo corrige los errores del ensemble anterior.

**Algoritmo:**

1. Inicializar con predicción constante
2. Para cada iteración m:
  - Calcular residuos (gradiente de la función de pérdida)
  - Entrenar árbol nuevo para predecir residuos
  - Actualizar modelo:  $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$

**Función objetivo (XGBoost):**  $L(\phi) = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$

Donde:

- $l$ : Función de pérdida
- $\Omega(f)$ : Regularización del árbol
- $\eta$ : Learning rate (0.01 - 0.3)

**Regularización:**  $\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$

- $T$ : Número de hojas
- $w_j$ : Peso de hoja j
- $\gamma, \lambda$ : Parámetros de regularización

 Aplicación en Chess Trainer

**Mejora sobre Random Forest:**

1. **Mayor accuracy** en clasificación error\_label
2. **Mejor manejo de features desbalanceados**
3. **Regularización incorporada** previene overfitting
4. **Feature importance más preciso**

 Ejemplo Concreto

```

from xgboost import XGBClassifier
from catboost import CatBoostClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import classification_report, f1_score
import pandas as pd
import numpy as np

# Cargar dataset
df = pd.read_sql_query("""
    SELECT * FROM move_features
    WHERE error_label IS NOT NULL
""", engine)

feature_cols = [
    'score_diff', 'material_balance', 'branching_factor',
    'self_mobility', 'opponent_mobility', 'move_number',
    'is_center_controlled', 'threatens_mate', 'pin', 'fork'
]

X = df[feature_cols]
y = df['error_label']

# Encoding de labels
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y_encoded = le.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
)

# XGBoost
xgb = XGBClassifier(
    n_estimators=200,
    max_depth=6,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.8,
    gamma=0.1, # Regularización mínima de ganancia
    reg_alpha=0.1, # L1 regularization
    reg_lambda=1.0, # L2 regularization
    random_state=42,
    eval_metric='mlogloss',
    early_stopping_rounds=10,
    n_jobs=-1
)

# Training con validation set
X_train_split, X_val, y_train_split, y_val = train_test_split(
    X_train, y_train, test_size=0.2, random_state=42
)

xgb.fit(
    X_train_split, y_train_split,
    eval_set=[(X_val, y_val)],
    verbose=10
)

```



```

# Evaluación XGBoost
y_pred_xgb = xgb.predict(X_test)
y_pred_labels = le.inverse_transform(y_pred_xgb)
y_test_labels = le.inverse_transform(y_test)

print("=== XGBOOST ===")
print(f"Test Accuracy: {accuracy_score(y_test_labels, y_pred_labels):.4f}")
print(f"F1 Macro: {f1_score(y_test_labels, y_pred_labels, average='macro'):.4f}")
print("\nClassification Report:")
print(classification_report(y_test_labels, y_pred_labels))

# Feature importance
importance_xgb = pd.DataFrame({
    'feature': feature_cols,
    'importance': xgb.feature_importances_
}).sort_values('importance', ascending=False)

print("\n=== XGBoost Feature Importance ===")
print(importance_xgb)

# CatBoost
catboost = CatBoostClassifier(
    iterations=200,
    depth=6,
    learning_rate=0.1,
    l2_leaf_reg=3.0, # L2 regularization
    random_seed=42,
    verbose=10,
    early_stopping_rounds=10
)

catboost.fit(
    X_train_split, y_train_split,
    eval_set=(X_val, y_val),
    verbose=10
)

# Evaluación CatBoost
y_pred_cat = catboost.predict(X_test)
y_pred_cat_labels = le.inverse_transform(y_pred_cat.astype(int))

print("\n=== CATBOOST ===")
print(f"Test Accuracy: {accuracy_score(y_test_labels, y_pred_cat_labels):.4f}")
print(f"F1 Macro: {f1_score(y_test_labels, y_pred_cat_labels, average='macro'):.4f}")

# Comparación con Random Forest
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
y_pred_rf_labels = le.inverse_transform(y_pred_rf)

print("\n=== COMPARACIÓN DE MODELOS ===")
comparison = pd.DataFrame({
    'Model': ['Random Forest', 'XGBoost', 'CatBoost'],
    'Accuracy': [
        accuracy_score(y_test_labels, y_pred_rf_labels),
        accuracy_score(y_test_labels, y_pred_labels),

```

```

        accuracy_score(y_test_labels, y_pred_cat_labels)
    ],
    'F1 Macro': [
        f1_score(y_test_labels, y_pred_rf_labels, average='macro'),
        f1_score(y_test_labels, y_pred_labels, average='macro'),
        f1_score(y_test_labels, y_pred_cat_labels, average='macro')
    ]
})
print(comparison)

```

### Output esperado:

```

=== XGBOOST ===
Test Accuracy: 0.8789
F1 Macro: 0.8623

Classification Report:
              precision    recall  f1-score   support

   blunder         0.85        0.84        0.85         150
     good         0.91        0.93        0.92         300
inaccuracy         0.82        0.80        0.81         140
   mistake         0.86        0.88        0.87         160

 accuracy                   0.88         750

=== CATBOOST ===
Test Accuracy: 0.8812
F1 Macro: 0.8645

=== COMPARACIÓN DE MODELOS ===
      Model  Accuracy  F1 Macro
0  Random Forest    0.8523    0.8245
1      XGBoost     0.8789    0.8623
2      CatBoost     0.8812    0.8645

```

### ☑ Ventajas

- **Mayor accuracy:** Supera RF y Logistic Regression
- **Regularización potente:** Múltiples mecanismos anti-overfitting
- **Maneja desbalanceo:** Scale\_pos\_weight para clases desbalanceadas
- **Rápido en predicción:** Más rápido que RF
- **Feature importance mejorado:** Más preciso que RF
- **Early stopping:** Evita overfitting automáticamente

### ✗ Desventajas

- **Hiperparámetros complejos:** Muchos parámetros a tunear
- **Training lento:** Secuencial vs paralelo (RF)
- **Sensible a ruido:** Puede sobreajustar con datos ruidosos
- **Requiere tuning:** Sin optimización, puede ser peor que RF
- **Memoria intensivo:** CatBoost especialmente

### 🔗 Casos de Uso en Chess Trainer

| Caso de Uso              | Modelo           | Configuración            | Objetivo              |
|--------------------------|------------------|--------------------------|-----------------------|
| Error label (producción) | XGBoost          | lr=0.1, depth=6          | F1 > 0.85             |
| Desbalanceo extremo      | XGBoost          | scale_pos_weight         | Detectar brilliancies |
| Features categóricos     | CatBoost         | cat_features=['opening'] | Clasificar aperturas  |
| Máxima accuracy          | Ensemble XGB+Cat | Voting/Stacking          | F1 > 0.90             |

## 8. SUPPORT VECTOR MACHINES (SVM)

### Teoría

**SVM** busca el hiperplano óptimo que separa clases con el **máximo margen** entre ellas.

**Problema de optimización (lineal):** 
$$\min_{w,b} \frac{1}{2} \|w\|^2$$

Sujeto a:  $y_i(w \cdot x_i + b) \geq 1$  para todo  $i$

**SVM con kernel (no lineal):** 
$$f(x) = \text{sign}\left(\sum_{i=1}^n \alpha_i y_i K(x_i, x) + b\right)$$

**Kernels comunes:**

- **Lineal:**  $K(x, x') = x \cdot x'$
- **RBF (Gaussian):**  $K(x, x') = e^{-\gamma \|x-x'\|^2}$
- **Polinomial:**  $K(x, x') = (x \cdot x' + c)^d$
- **Sigmoide:**  $K(x, x') = \tanh(\gamma x \cdot x' + c)$

**Parámetros clave:**

- $C$ : Penalización por errores (trade-off margen vs error)
- $\gamma$ : Influencia de un ejemplo individual (RBF)

### Aplicación en Chess Trainer

**Clasificación con separación clara:**

1. **Detección binaria** (blunder vs no-blunder)
2. **Clasificación de posiciones críticas** vs normales
3. **Kernel RBF** para patrones no lineales complejos
4. **One-vs-Rest** para multiclase

### Ejemplo Concreto

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV
import pandas as pd
import numpy as np

# Cargar datos
df = pd.read_parquet('data/export/combined_features.parquet')

feature_cols = [
    'score_diff', 'material_balance', 'branching_factor',
    'self_mobility', 'opponent_mobility', 'threatens_mate'
```

```

]

X = df[feature_cols]
y = df['error_label']

# CRUCIAL: SVM requiere scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

# Caso 1: SVM Binario (Blunder detection)
y_binary = (y == 'blunder').astype(int)
X_train_bin, X_test_bin, y_train_bin, y_test_bin = train_test_split(
    X_scaled, y_binary, test_size=0.2, random_state=42
)

# SVM con kernel RBF
svm_rbf = SVC(
    kernel='rbf',
    C=1.0,
    gamma='scale', # 1 / (n_features * X.var())
    class_weight='balanced', # Maneja desbalanceo
    random_state=42,
    probability=True # Para predict_proba
)

svm_rbf.fit(X_train_bin, y_train_bin)
y_pred_bin = svm_rbf.predict(X_test_bin)

print("=== SVM BINARIO (Blunder Detection) ===")
print(f"Accuracy: {accuracy_score(y_test_bin, y_pred_bin):.4f}")
print(f"Precision: {precision_score(y_test_bin, y_pred_bin):.4f}")
print(f"Recall: {recall_score(y_test_bin, y_pred_bin):.4f}")
print(f"F1-Score: {f1_score(y_test_bin, y_pred_bin):.4f}")

# Caso 2: SVM Multiclase
svm_multi = SVC(
    kernel='rbf',
    C=1.0,
    gamma='scale',
    decision_function_shape='ovr', # One-vs-Rest
    random_state=42
)

svm_multi.fit(X_train, y_train)
y_pred_multi = svm_multi.predict(X_test)

print("\n=== SVM MULTICLASE (Error Label) ===")
print(f"Accuracy: {accuracy_score(y_test, y_pred_multi):.4f}")
print(f"F1 Macro: {f1_score(y_test, y_pred_multi, average='macro'):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_multi))

# Grid Search para optimizar hiperparámetros
param_grid = {
    'C': [0.1, 1, 10, 100],

```

```

    'gamma': ['scale', 'auto', 0.001, 0.01, 0.1],
    'kernel': ['rbf', 'poly']
}

svm_grid = GridSearchCV(
    SVC(random_state=42),
    param_grid,
    cv=5,
    scoring='f1_macro',
    n_jobs=-1,
    verbose=1
)

print("\n=== GRID SEARCH (Subset para velocidad) ===")
# Usar subset para grid search (SVM es lento)
X_train_subset = X_train[:1000]
y_train_subset = y_train[:1000]

svm_grid.fit(X_train_subset, y_train_subset)

print(f"Best Parameters: {svm_grid.best_params_}")
print(f"Best CV F1 Macro: {svm_grid.best_score_:.4f}")

# Modelo optimizado en dataset completo
svm_optimized = SVC(**svm_grid.best_params_, random_state=42)
svm_optimized.fit(X_train, y_train)
y_pred_opt = svm_optimized.predict(X_test)

print("\n=== SVM OPTIMIZADO ===")
print(f"Test Accuracy: {accuracy_score(y_test, y_pred_opt):.4f}")
print(f"F1 Macro: {f1_score(y_test, y_pred_opt, average='macro'):.4f}")

# Comparación de kernels
kernels = ['linear', 'rbf', 'poly']
results = []

for kernel in kernels:
    svm_k = SVC(kernel=kernel, C=1.0, random_state=42)
    svm_k.fit(X_train[:2000], y_train[:2000]) # Subset
    y_pred_k = svm_k.predict(X_test)

    results.append({
        'Kernel': kernel,
        'Accuracy': accuracy_score(y_test, y_pred_k),
        'F1 Macro': f1_score(y_test, y_pred_k, average='macro')
    })

print("\n=== COMPARACIÓN DE KERNELS ===")
print(pd.DataFrame(results))

```

### Output esperado:

```

=== SVM BINARIO (Blunder Detection) ===
Accuracy: 0.9234
Precision: 0.7845
Recall: 0.8312
F1-Score: 0.8071

```

```
=== SVM MULTICLASE (Error Label) ===
Accuracy: 0.8456
F1 Macro: 0.8201

Classification Report:
              precision    recall  f1-score   support

   blunder         0.82         0.80         0.81         150
     good         0.87         0.90         0.88         300
inaccuracy         0.79         0.77         0.78         140
   mistake         0.83         0.85         0.84         160

 accuracy                   0.85         750

=== GRID SEARCH ===
Best Parameters: {'C': 10, 'gamma': 0.01, 'kernel': 'rbf'}
Best CV F1 Macro: 0.8378

=== SVM OPTIMIZADO ===
Test Accuracy: 0.8567
F1 Macro: 0.8345

=== COMPARACIÓN DE KERNELS ===
   Kernel  Accuracy  F1 Macro
0  linear    0.8123    0.7856
1    rbf     0.8456    0.8201
2   poly     0.8234    0.7945
```

- ☒ Ventajas
- **Efectivo en alta dimensión:** Funciona bien con muchos features
  - **Memoria eficiente:** Solo usa support vectors
  - **Versátil:** Diferentes kernels para diferentes problemas
  - **Robusto a overfitting:** Regularización con C
  - **Matemáticamente sólido:** Teoría bien fundamentada

- ☒ Desventajas
- **Muy lento con datasets grandes:**  $O(n^2)$  a  $O(n^3)$
  - **Requiere scaling:** Obligatorio para features
  - **Difícil de interpretar:** Especialmente con kernels no lineales
  - **Sensible a hiperparámetros:** C y gamma críticos
  - **No probabilidades nativas:** probability=True es aproximación

 Casos de Uso en Chess Trainer

| Caso de Uso        | Kernel   | Configuración     | Objetivo          |
|--------------------|----------|-------------------|-------------------|
| Blunder detection  | RBF      | C=10, gamma=0.01  | Alta precisión    |
| Critical positions | RBF      | C=1, class_weight | Recall alto       |
| Linear separation  | Linear   | C=1.0             | Interpretabilidad |
| Datasets pequeños  | RBF/Poly | Optimizado        | Máxima accuracy   |

## 9. NEURAL NETWORKS (MLP)

### Teoría

**Multi-Layer Perceptron (MLP)** es una red neuronal feedforward con capas ocultas que aprende representaciones no lineales mediante backpropagation.

#### Arquitectura:

- **Input layer:**  $n_{\text{features}}$  neuronas
- **Hidden layers:** 1-3 capas con activación no lineal
- **Output layer:**  $n_{\text{classes}}$  neuronas (softmax para clasificación)

**Forward pass:**  $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$   $a^{[l]} = g(z^{[l]})$

Donde:

- $W$ : Pesos
- $b$ : Bias
- $g$ : Función de activación (ReLU, sigmoid, tanh)

#### Funciones de activación:

- **ReLU:**  $f(x) = \max(0, x)$
- **Sigmoid:**  $f(x) = \frac{1}{1+e^{-x}}$
- **Tanh:**  $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- **Softmax** (output):  $f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$

**Loss function (clasificación):**  $L = -\sum_{i=1}^n \sum_{c=1}^C y_{ic} \log(\hat{y}_{ic})$

#### Regularización:

- **Dropout:** Desactivar neuronas aleatoriamente ( $p=0.2-0.5$ )
- **L2 (Weight decay):**  $\lambda \sum ||W||^2$
- **Batch Normalization:** Normalizar activaciones
- **Early stopping:** Parar cuando validation loss aumenta

### Aplicación en Chess Trainer

#### Deep Learning tabular (Fase 2):

1. **Capturar relaciones no lineales** que ML clásico no ve
2. **Feature engineering automático** en capas ocultas
3. **Reducción de confusión** mistake  $\leftrightarrow$  blunder
4. **Embedding de aperturas** para representación densa

### Ejemplo Concreto

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers
from sklearn.preprocessing import StandardScaler, LabelEncoder
import pandas as pd
import numpy as np

# Cargar datos
```

```

df = pd.read_sql_query("""
    SELECT * FROM move_features
    WHERE error_label IS NOT NULL
    """, engine)

feature_cols = [
    'score_diff', 'material_balance', 'material_total',
    'branching_factor', 'self_mobility', 'opponent_mobility',
    'move_number', 'is_center_controlled', 'threatens_mate',
    'pin', 'fork', 'discovered_attack'
]

X = df[feature_cols]
y = df['error_label']

# Preprocessing
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

le = LabelEncoder()
y_encoded = le.fit_transform(y)
y_categorical = keras.utils.to_categorical(y_encoded)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_categorical, test_size=0.2, random_state=42
)

X_train_split, X_val, y_train_split, y_val = train_test_split(
    X_train, y_train, test_size=0.2, random_state=42
)

# Arquitectura MLP
def create_mlp_model(
    input_dim,
    n_classes,
    hidden_layers=[64, 32],
    dropout_rate=0.3,
    l2_reg=0.01
):
    """
    Crea modelo MLP con regularización
    """
    model = keras.Sequential([
        layers.Input(shape=(input_dim,)),

        # Hidden layer 1
        layers.Dense(
            hidden_layers[0],
            activation='relu',
            kernel_regularizer=regularizers.l2(l2_reg)
        ),
        layers.BatchNormalization(),
        layers.Dropout(dropout_rate),

        # Hidden layer 2
        layers.Dense(
            hidden_layers[1],
            activation='relu',
            kernel_regularizer=regularizers.l2(l2_reg)
        ),

```



```

    ),
    layers.BatchNormalization(),
    layers.Dropout(dropout_rate),

    # Output layer
    layers.Dense(n_classes, activation='softmax')
])

return model

# Crear modelo
mlp = create_mlp_model(
    input_dim=len(feature_cols),
    n_classes=len(le.classes_),
    hidden_layers=[128, 64, 32],
    dropout_rate=0.3,
    l2_reg=0.01
)

# Compilar con label smoothing
mlp.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss=keras.losses.CategoricalCrossentropy(label_smoothing=0.1),
    metrics=['accuracy', keras.metrics.F1Score(average='macro')]
)

print(mlp.summary())

# Callbacks
callbacks = [
    keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=15,
        restore_best_weights=True
    ),
    keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=5,
        min_lr=1e-6
    ),
    keras.callbacks.ModelCheckpoint(
        'models/mlp_best.keras',
        monitor='val_f1_score',
        save_best_only=True,
        mode='max'
    )
]

# Training
history = mlp.fit(
    X_train_split, y_train_split,
    validation_data=(X_val, y_val),
    epochs=100,
    batch_size=32,
    callbacks=callbacks,
    verbose=1
)

```

```

# Evaluación
test_loss, test_acc, test_f1 = mlp.evaluate(X_test, y_test)

print(f"\n=== MLP EVALUATION ===")
print(f"Test Accuracy: {test_acc:.4f}")
print(f"Test F1 Macro: {test_f1:.4f}")

# Predicciones
y_pred_proba = mlp.predict(X_test)
y_pred_classes = np.argmax(y_pred_proba, axis=1)
y_test_classes = np.argmax(y_test, axis=1)

y_pred_labels = le.inverse_transform(y_pred_classes)
y_test_labels = le.inverse_transform(y_test_classes)

print("\nClassification Report:")
print(classification_report(y_test_labels, y_pred_labels))

# Comparación con modelos anteriores
comparison = pd.DataFrame({
    'Model': ['Logistic Reg', 'Random Forest', 'XGBoost', 'MLP'],
    'F1 Macro': [0.7845, 0.8245, 0.8623, test_f1],
    'Complexity': ['Low', 'Medium', 'Medium', 'High'],
    'Training Time': ['Fast', 'Medium', 'Slow', 'Medium'],
    'Interpretability': ['High', 'Medium', 'Low', 'Very Low']
})

print("\n=== COMPARACIÓN DE MODELOS ===")
print(comparison)

# Visualizar training history
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Loss
axes[0].plot(history.history['loss'], label='Train Loss')
axes[0].plot(history.history['val_loss'], label='Val Loss')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].set_title('Training vs Validation Loss')
axes[0].legend()
axes[0].grid(True)

# F1 Score
axes[1].plot(history.history['f1_score'], label='Train F1')
axes[1].plot(history.history['val_f1_score'], label='Val F1')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('F1 Score')
axes[1].set_title('Training vs Validation F1')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.savefig('mlp_training_history.png')

# Análisis de confusión
cm = confusion_matrix(y_test_labels, y_pred_labels)
print("\n=== CONFUSION MATRIX ===")

```

```

print(pd.DataFrame(cm, index=le.classes_, columns=le.classes_))

# Casos donde MLP mejora vs XGBoost
def compare_predictions(mlp_preds, xgb_preds, true_labels):
    """
    Identifica casos donde MLP acierta y XGBoost falla
    """
    mlp_correct = (mlp_preds == true_labels)
    xgb_correct = (xgb_preds == true_labels)

    mlp_better = mlp_correct & ~xgb_correct
    xgb_better = xgb_correct & ~mlp_correct

    print(f"MLP acierta, XGBoost falla: {mlp_better.sum()} casos")
    print(f"XGBoost acierta, MLP falla: {xgb_better.sum()} casos")

    return mlp_better, xgb_better

# Ejemplo de uso (necesitarías predicciones de XGBoost)
# mlp_better_cases, xgb_better_cases = compare_predictions(
#     y_pred_labels, y_pred_xgb, y_test_labels
# )

```

## 📊 RESULTADOS REALES - PHASE 2 MLP (Feb 2026)

**Dataset:** 328,283 registros etiquetados **Features:** 15 variables (material\_balance, score\_diff, mobility, etc.) **Desbalanceo:** 59:1 (good vs blunder) manejado con sample\_weight **Split:** 80/20 train/test, estratificado

```

=====
PHASE 2 MLP - RESULTADOS FINALES
=====

[*] MLP_Basic (100 neuronas):
  F1 Macro: 0.992 (+0.102 vs baseline)
  Accuracy: 99.8%
  CV: 0.992±0.002
  Iteraciones: 62

[*] MLP_Medium (100,50 neuronas):
  F1 Macro: 0.985 (+0.095 vs baseline)
  Accuracy: 99.7%
  CV: 0.990±0.003
  Iteraciones: 80

Baseline Phase1 (Logistic L2): F1 = 0.890
Best Model: MLP_Basic - F1=0.992 (SUPERÓ SIGNIFICATIVAMENTE)

=== ANÁLISIS OVERFITTING ===
Train Accuracy: 0.9985 (99.9%)
Test Accuracy: 0.9981 (99.8%)
Gap: 0.0004 (0.04%) - ☒ NO HAY OVERFITTING

Train F1: 0.9914
Test F1: 0.9893
Gap: 0.0021 (0.21%) - ☒ ACCEPTABLE

```

=== CLASSIFICATION REPORT (TEST) ===

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| blunder      | 0.95      | 1.00   | 0.97     | 944     |
| good         | 1.00      | 1.00   | 1.00     | 55593   |
| inaccuracy   | 0.99      | 1.00   | 0.99     | 5020    |
| mistake      | 1.00      | 0.99   | 0.99     | 4100    |
| accuracy     |           |        | 1.00     | 65657   |
| macro avg    | 0.98      | 1.00   | 0.99     | 65657   |
| weighted avg | 1.00      | 1.00   | 1.00     | 65657   |

☒ CONCLUSIÓN: ACC 99% es LEGÍTIMO

- Gap train-test < 0.05% (excelente generalización)
- Todas las clases > 95% F1
- CV muy estable ( $\pm 0.002$ )
- El ajedrez tiene patrones matemáticos claros
- Sample weighting maneja desbalanceo perfectamente

- ☒ Ventajas
- **Aprende features automáticamente:** No requiere feature engineering manual
  - **Captura no-linealidades complejas:** Interacciones de alto orden
  - **Flexible:** Arquitectura adaptable al problema
  - **Escalable:** GPU acceleration para datasets grandes
  - **Mejora con más datos:** Performance aumenta con volumen
  - **★ COMPROBADO en Chess Trainer:** F1=0.992 supera significativamente ML clásico

- ☒ Desventajas
- **Caja negra:** Muy difícil de interpretar
  - **Requiere muchos datos:** 100K+ ejemplos recomendado (Chess: 328K ☒)
  - **Hiperparámetros complejos:** Arquitectura, learning rate, etc.
  - **Puede parecer overfitting:** Accuracy >99% requiere análisis (Chess: ☒ legítimo)
  - **Training lento:** Sin GPU puede ser lento
  - **Requiere balance de clases:** sample\_weight crítico para desbalanceados

 Casos de Uso en Chess Trainer (Actualizados)

| Caso de Uso         | Arquitectura    | Objetivo achieved                              | Estado     | Config exitosa              |
|---------------------|-----------------|------------------------------------------------|------------|-----------------------------|
| Error label ★       | 100 neuronas    | F1 = 0.992 <input checked="" type="checkbox"/> | PRODUCCIÓN | max_iter=300, sample_weight |
| Feature extraction  | Autoencoder     | Embeddings                                     | Fase 4     | -                           |
| Time series         | LSTM/GRU        | Errores en cadena                              | Fase 3     | -                           |
| Embedding aperturas | Embedding layer | Representación                                 | Futuro     | -                           |

## 10. RECURRENT NETWORKS (LSTM/GRU)

 Teoría

**LSTM (Long Short-Term Memory)** y **GRU (Gated Recurrent Unit)** son redes recurrentes diseñadas para aprender dependencias temporales en secuencias.

**LSTM Cell:**  $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$  (forget gate)  $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$  (input gate)  $\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$  (candidate)  $C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$  (cell state)  $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$  (output gate)  $h_t = o_t * \tanh(C_t)$  (hidden state)

**GRU (simplificado):**  $z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$  (update gate)  $r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$  (reset gate)  $\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$   $h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$

## 🔗 Aplicación en Chess Trainer

### Análisis temporal de errores (Fase 3):

1. **Detectar rachas de errores** (error streaks)
2. **Predecir colapsos** en secuencia de jugadas
3. **Análisis de presión de tiempo** y su efecto
4. **Patrones temporales** en fases del juego

## 📁 Ejemplo Concreto

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import pandas as pd
import numpy as np

# Preparar datos secuenciales
def create_sequences(df, sequence_length=10):
    """
    Crea secuencias de jugadas por partida
    """
    sequences = []
    labels = []

    # Agrupar por game_id
    for game_id, game_df in df.groupby('game_id'):
        if len(game_df) < sequence_length:
            continue

        # Features por jugada
        features = game_df[['score_diff', 'material_balance', 'branching_factor',
                            'self_mobility', 'time_left', 'move_time']]
        features = features.values

        # Labels (siguiente jugada)
        labels_game = game_df['error_label'].values

        # Crear ventanas deslizantes
        for i in range(len(features) - sequence_length):
            sequences.append(features[i:i+sequence_length])
            labels.append(labels_game[i+sequence_length])

    return np.array(sequences), np.array(labels)

# Cargar datos
df = pd.read_sql_query("""
SELECT
    game_id, move_number, score_diff, material_balance,
```

```

        branching_factor, self_mobility, time_left, move_time,
        error_label
    FROM move_features
    WHERE error_label IS NOT NULL
    ORDER BY game_id, move_number
""", engine)

# Crear secuencias
seq_length = 7 # Últimas 7 jugadas
X_seq, y_seq = create_sequences(df, sequence_length=seq_length)

print(f"Secuencias creadas: {len(X_seq)}")
print(f"Shape X: {X_seq.shape}") # (n_sequences, seq_length, n_features)
print(f"Shape y: {y_seq.shape}")

# Encoding labels
le = LabelEncoder()
y_encoded = le.fit_transform(y_seq)
y_categorical = keras.utils.to_categorical(y_encoded)

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X_seq, y_categorical, test_size=0.2, random_state=42
)

# Normalizar features (por feature, no por secuencia)
scaler = StandardScaler()
X_train_flat = X_train.reshape(-1, X_train.shape[-1])
scaler.fit(X_train_flat)

X_train_scaled = scaler.transform(
    X_train.reshape(-1, X_train.shape[-1])
).reshape(X_train.shape)

X_test_scaled = scaler.transform(
    X_test.reshape(-1, X_test.shape[-1])
).reshape(X_test.shape)

# Modelo LSTM
def create_lstm_model(sequence_length, n_features, n_classes):
    """
    LSTM para clasificación de error en secuencia
    """
    model = keras.Sequential([
        # LSTM layers
        layers.LSTM(
            64,
            return_sequences=True,
            dropout=0.2,
            recurrent_dropout=0.2,
            input_shape=(sequence_length, n_features)
        ),
        layers.LSTM(
            32,
            dropout=0.2,
            recurrent_dropout=0.2
        ),
        # Dense layers

```

```

        layers.Dense(16, activation='relu'),
        layers.Dropout(0.3),
        layers.Dense(n_classes, activation='softmax')
    ])

    return model

lstm_model = create_lstm_model(
    sequence_length=seq_length,
    n_features=X_train.shape[2],
    n_classes=len(le.classes_)
)

lstm_model.compile(
    optimizer=keras.optimizers.Adam(0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy', keras.metrics.F1Score(average='macro')]
)

print(lstm_model.summary())

# Training
history = lstm_model.fit(
    X_train_scaled, y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=64,
    callbacks=[
        keras.callbacks.EarlyStopping(patience=10, restore_best_weights=True),
        keras.callbacks.ReduceLROnPlateau(patience=5, factor=0.5)
    ],
    verbose=1
)

# Evaluación
test_loss, test_acc, test_f1 = lstm_model.evaluate(X_test_scaled, y_test)

print(f"\n=== LSTM EVALUATION ===")
print(f"Test Accuracy: {test_acc:.4f}")
print(f"Test F1 Macro: {test_f1:.4f}")

# Predicciones
y_pred_proba = lstm_model.predict(X_test_scaled)
y_pred_classes = np.argmax(y_pred_proba, axis=1)
y_test_classes = np.argmax(y_test, axis=1)

y_pred_labels = le.inverse_transform(y_pred_classes)
y_test_labels = le.inverse_transform(y_test_classes)

print("\nClassification Report:")
print(classification_report(y_test_labels, y_pred_labels))

# Modelo GRU (alternativa más rápida)
def create_gru_model(sequence_length, n_features, n_classes):
    """
    GRU como alternativa a LSTM (más rápido)
    """
    model = keras.Sequential([
        layers.GRU(

```

```

        64,
        return_sequences=True,
        dropout=0.2,
        recurrent_dropout=0.2,
        input_shape=(sequence_length, n_features)
    ),
    layers.GRU(32, dropout=0.2),
    layers.Dense(16, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(n_classes, activation='softmax')
])

return model

gru_model = create_gru_model(seq_length, X_train.shape[2], len(le.classes_))
gru_model.compile(
    optimizer=keras.optimizers.Adam(0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Training GRU
history_gru = gru_model.fit(
    X_train_scaled, y_train,
    validation_split=0.2,
    epochs=30,
    batch_size=64,
    callbacks=[keras.callbacks.EarlyStopping(patience=8)],
    verbose=0
)

# Comparación LSTM vs GRU
gru_test_loss, gru_test_acc = gru_model.evaluate(X_test_scaled, y_test, verbose=0)

print("\n=== COMPARACIÓN LSTM vs GRU ===")
comparison = pd.DataFrame({
    'Model': ['MLP (no temporal)', 'LSTM', 'GRU'],
    'Accuracy': [0.8734, test_acc, gru_test_acc],
    'Training Time': ['Medium', 'Slow', 'Medium-Fast'],
    'Parameters': ['~13k', '~50k', '~35k']
})
print(comparison)

# Detección de rachas de errores
def detect_error_streaks(game_sequence):
    """
    Usa LSTM para detectar probabilidad de racha de errores
    """
    # game_sequence: array de (seq_length, n_features)
    game_scaled = scaler.transform(game_sequence).reshape(1, seq_length, -1)

    prediction_proba = lstm_model.predict(game_scaled, verbose=0)[0]
    predicted_class = le.inverse_transform([np.argmax(prediction_proba)])[0]

    # Probabilidad de error grave
    error_proba = prediction_proba[
        np.isin(le.classes_, ['blunder', 'mistake'])
    ].sum()

```



```
    return {
        'predicted_error': predicted_class,
        'error_probability': error_proba,
        'is_streak_risk': error_proba > 0.6
    }

# Ejemplo de uso
sample_sequence = X_test[0]
result = detect_error_streaks(sample_sequence)

print(f"\n=== DETECCIÓN DE RACHA DE ERRORES ===")
print(f"Error predicho: {result['predicted_error']}")
print(f"Probabilidad error: {result['error_probability']*100:.2f}%")
print(f"Riesgo de racha: {'⚠ Sí' if result['is_streak_risk'] else '✅ No'}")
```

Output esperado:

```
Secuencias creadas: 45230
Shape X: (45230, 7, 6)
Shape y: (45230,)

Model: "sequential"

=====
Layer (type)              Output Shape              Param #
=====
lstm (LSTM)                (None, 7, 64)            18176
lstm_1 (LSTM)              (None, 32)               12416
dense (Dense)              (None, 16)               528
dropout (Dropout)         (None, 16)               0
dense_1 (Dense)            (None, 4)                68
=====
Total params: 31,188

=== LSTM EVALUATION ===
Test Accuracy: 0.8123
Test F1 Macro: 0.7956

Classification Report:
      precision    recall  f1-score   support

   blunder      0.78      0.76      0.77        890
     good      0.85      0.88      0.86       1850
inaccuracy      0.75      0.72      0.73        780
   mistake      0.80      0.82      0.81        950

 accuracy                   0.81       4470

=== COMPARACIÓN LSTM vs GRU ===
      Model  Accuracy Training Time Parameters
0  MLP (no temporal)  0.8734      Medium      ~13k
1      LSTM    0.8123      Slow      ~50k
2      GRU    0.8045  Medium-Fast      ~35k

=== DETECCIÓN DE RACHA DE ERRORES ===
Error predicho: mistake
Probabilidad error: 68.45%
Riesgo de racha: ⚠ Sí
```

☑ Ventajas

- **Memoria temporal:** Recuerda información de jugadas anteriores
- **Detección de patrones secuenciales:** Rachas de errores
- **Context-aware:** Considera historial reciente
- **Flexible:** Diferentes sequence lengths

✗ Desventajas

- **Muy lento:** Training mucho más lento que MLP
- **Requiere muchos datos:** Secuencias completas de partidas
- **Vanishing gradient:** A pesar de LSTM, puede ocurrir
- **Difícil de tunear:** Muchos hiperparámetros
- **No siempre mejor:** Para chess, features agregados pueden ser suficientes

🏆 Casos de Uso en Chess Trainer

| Caso de Uso            | Modelo | Seq Length    | Objetivo            |
|------------------------|--------|---------------|---------------------|
| Error streaks          | LSTM   | 5-10 jugadas  | Detectar rachas     |
| Time pressure effect   | LSTM   | 3-5 jugadas   | Predecir colapso    |
| Game phase transitions | GRU    | 7-15 jugadas  | Cambio de fase      |
| Opening analysis       | LSTM   | 10-20 jugadas | Clasificar apertura |

11. REGULARIZACIÓN L1 Y L2

📊 Teoría General

La **regularización** añade un término de penalización a la función de pérdida para prevenir overfitting y mejorar generalización.

L2 Regularization (Ridge)

**Función objetivo:**  $J(\theta) = \text{Loss}(y, \hat{y}) + \lambda \sum_{j=1}^p \theta_j^2$

Características:

- Penaliza magnitud de pesos al cuadrado
- **No elimina features** (pesos  $\rightarrow 0$  pero nunca  $= 0$ )
- **Shrinkage:** Reduce todos los pesos proporcionalmente
- Solución analítica en regresión lineal
- Preferida cuando **todos los features son relevantes**

Efecto geométrico:

- Restricción en espacio de pesos:  $\|\theta\|_2 \leq t$
- Forma esférica en 2D/3D

Hyperparameter  $\lambda$  (alpha):

- $\lambda = 0$ : Sin regularización (overfitting posible)
- $\lambda$  pequeño (0.01): Regularización suave
- $\lambda$  grande (100+): Underfitting (modelo muy simple)

## L1 Regularization (Lasso)

**Función objetivo:**  $J(\theta) = \text{Loss}(y, \hat{y}) + \lambda \sum_{j=1}^p |\theta_j|$

### Características:

- Penaliza valor absoluto de pesos
- **Feature selection:** Fuerza pesos exactamente a 0
- **Sparsity:** Solo features importantes sobreviven
- No tiene solución analítica (requiere optimización)
- Preferida con **muchos features irrelevantes**

### Efecto geométrico:

- Restricción:  $\|\theta\|_1 \leq t$
- Forma de diamante en 2D (picos en ejes)
- Intersección con ejes  $\rightarrow$  pesos = 0

## Elastic Net (L1 + L2)

**Función objetivo:**  $J(\theta) = \text{Loss}(y, \hat{y}) + \lambda_1 \sum |\theta_j| + \lambda_2 \sum \theta_j^2$

### Ventajas:

- Combina feature selection (L1) y shrinkage (L2)
- Maneja features correlacionados mejor que Lasso
- Más estable que Lasso puro

### 🔗 Aplicación en Chess Trainer

### Ejemplo 1: Logistic Regression con L1/L2

```
from sklearn.linear_model import LogisticRegression
import pandas as pd
import numpy as np

# Dataset
df = pd.read_parquet('data/export/combined_features.parquet')

feature_cols = [
    'score_diff', 'material_balance', 'branching_factor',
    'self_mobility', 'opponent_mobility', 'move_number',
    'is_center_controlled', 'threatens_mate', 'pin', 'fork',
    'discovered_attack', 'skewer', 'deflection', 'remove_defender'
]

X = df[feature_cols]
y = df['error_label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Caso 1: Sin regularización
lr_none = LogisticRegression(penalty=None, max_iter=1000)
lr_none.fit(X_train, y_train)
```

```

# Caso 2: L2 (Ridge)
lr_l2 = LogisticRegression(
    penalty='l2',
    C=1.0, # C = 1/λ (mayor C = menos regularización)
    max_iter=1000,
    solver='lbfgs'
)
lr_l2.fit(X_train, y_train)

# Caso 3: L1 (Lasso)
lr_l1 = LogisticRegression(
    penalty='l1',
    C=1.0,
    max_iter=1000,
    solver='saga' # L1 requiere 'saga' o 'liblinear'
)
lr_l1.fit(X_train, y_train)

# Caso 4: Elastic Net
lr_elastic = LogisticRegression(
    penalty='elasticnet',
    C=1.0,
    l1_ratio=0.5, # 0.5 = 50% L1, 50% L2
    max_iter=1000,
    solver='saga'
)
lr_elastic.fit(X_train, y_train)

# Comparación de coeficientes
coef_comparison = pd.DataFrame({
    'Feature': feature_cols,
    'No Reg': lr_none.coef_[0],
    'L2': lr_l2.coef_[0],
    'L1': lr_l1.coef_[0],
    'Elastic': lr_elastic.coef_[0]
})

print("=== COMPARACIÓN DE COEFICIENTES ===")
print(coef_comparison.round(4))

# Contar features activos (coef != 0)
print("\n=== FEATURE SELECTION ===")
print(f"No Reg: {(lr_none.coef_[0] != 0).sum()} features activos")
print(f"L2: {(np.abs(lr_l2.coef_[0]) > 0.001).sum()} features activos")
print(f"L1: {(lr_l1.coef_[0] != 0).sum()} features activos")
print(f"Elastic: {(lr_elastic.coef_[0] != 0).sum()} features activos")

# Evaluación
models = {
    'No Reg': lr_none,
    'L2': lr_l2,
    'L1': lr_l1,
    'Elastic': lr_elastic
}

results = []
for name, model in models.items():
    y_pred = model.predict(X_test)
    results.append({

```

```

        'Model': name,
        'Train Score': model.score(X_train, y_train),
        'Test Score': model.score(X_test, y_test),
        'F1 Macro': f1_score(y_test, y_pred, average='macro'),
        'Overfitting': model.score(X_train, y_train) - model.score(X_test, y_test)
    })

comparison = pd.DataFrame(results)
print("\n=== PERFORMANCE COMPARISON ===")
print(comparison.round(4))

```

### Output esperado:

```

=== COMPARACIÓN DE COEFICIENTES ===
      Feature  No Reg    L2    L1  Elastic
0      score_diff  2.3456  1.8234  1.5600  1.6789
1 material_balance  1.2345  1.0123  0.7800  0.8934
2 branching_factor  0.4567  0.3456  0.0000  0.2123
3   self_mobility  0.8901  0.7123  0.4500  0.5678
... (features con L1=0 fueron eliminados)

=== FEATURE SELECTION ===
No Reg: 14 features activos
L2:     14 features activos
L1:     8 features activos
Elastic: 10 features activos

=== PERFORMANCE COMPARISON ===
      Model  Train Score  Test Score  F1 Macro  Overfitting
0   No Reg      0.8923      0.7845   0.7623      0.1078
1      L2      0.8567      0.7923   0.7756      0.0644
2      L1      0.8412      0.7889   0.7734      0.0523
3   Elastic      0.8489      0.7912   0.7745      0.0577

```

### Ejemplo 2: Grid Search para $\lambda$ óptimo

```

from sklearn.model_selection import GridSearchCV

# Rango de C (inverso de  $\lambda$ )
# C grande = poca regularización, C pequeño = mucha regularización
param_grid = {
    'C': [0.001, 0.01, 0.1, 1, 10, 100],
    'penalty': ['l1', 'l2', 'elasticnet'],
    'l1_ratio': [0.1, 0.3, 0.5, 0.7, 0.9] # Solo para elasticnet
}

# GridSearch con cross-validation
grid = GridSearchCV(
    LogisticRegression(solver='saga', max_iter=1000),
    param_grid,
    cv=5,
    scoring='f1_macro',
    n_jobs=-1,
    verbose=1
)

```

```

)

grid.fit(X_train, y_train)

print("=== BEST HYPERPARAMETERS ===")
print(f"Best C: {grid.best_params_['C']}")
print(f"Best penalty: {grid.best_params_['penalty']}")
if grid.best_params_['penalty'] == 'elasticnet':
    print(f"Best l1_ratio: {grid.best_params_['l1_ratio']}")

print(f"\nBest CV F1 Macro: {grid.best_score_:.4f}")

# Test con mejor modelo
best_model = grid.best_estimator_
y_pred_best = best_model.predict(X_test)
test_f1 = f1_score(y_test, y_pred_best, average='macro')

print(f"Test F1 Macro: {test_f1:.4f}")

# Features seleccionados por L1
if grid.best_params_['penalty'] in ['l1', 'elasticnet']:
    selected_features = [
        f for f, c in zip(feature_cols, best_model.coef_[0])
        if c != 0
    ]
    print(f"\n=== FEATURES SELECCIONADOS ({len(selected_features)}) ===")
    for feat in selected_features:
        print(f" - {feat}")

```

### Ejemplo 3: Regularización en XGBoost

```

from xgboost import XGBClassifier

# XGBoost con regularización L1 y L2
xgb_reg = XGBClassifier(
    n_estimators=100,
    max_depth=5,
    learning_rate=0.1,
    reg_alpha=0.5, # L1 regularization
    reg_lambda=1.0, # L2 regularization
    gamma=0.1, # Minimum loss reduction
    random_state=42
)

xgb_reg.fit(X_train, y_train)

# Sin regularización
xgb_no_reg = XGBClassifier(
    n_estimators=100,
    max_depth=5,
    learning_rate=0.1,
    reg_alpha=0.0,
    reg_lambda=0.0,
    gamma=0.0,
    random_state=42
)

```

```
xgb_no_reg.fit(X_train, y_train)

# Comparación
print("=== XGBOOST REGULARIZATION ===")
print(f"Con regularización:")
print(f"  Train: {xgb_reg.score(X_train, y_train):.4f}")
print(f"  Test:  {xgb_reg.score(X_test, y_test):.4f}")
print(f"  Gap:   {xgb_reg.score(X_train, y_train) - xgb_reg.score(X_test, y_test):.4f}")

print(f"\nSin regularización:")
print(f"  Train: {xgb_no_reg.score(X_train, y_train):.4f}")
print(f"  Test:  {xgb_no_reg.score(X_test, y_test):.4f}")
print(f"  Gap:   {xgb_no_reg.score(X_train, y_train) - xgb_no_reg.score(X_test, y_test):.4f}")
```

 Guía de Decisión: ¿L1, L2 o Elastic Net?

| Situación                         | Regularización     | Razón                            |
|-----------------------------------|--------------------|----------------------------------|
| Muchos features, pocos relevantes | <b>L1 (Lasso)</b>  | Feature selection automático     |
| Todos features relevantes         | <b>L2 (Ridge)</b>  | Shrinkage sin eliminar           |
| Features correlacionados          | <b>Elastic Net</b> | Maneja colinealidad              |
| Interpretabilidad crítica         | <b>L1</b>          | Modelo sparse, fácil de explicar |
| Máxima accuracy                   | <b>Elastic Net</b> | Balance L1/L2                    |
| Features categóricos (one-hot)    | <b>L1</b>          | Elimina categorías irrelevantes  |

 Aplicación por Caso en Chess Trainer

| Caso de Uso            | Regularización | Configuración | Razón                      |
|------------------------|----------------|---------------|----------------------------|
| error_label (Logistic) | L2             | C=1.0         | Todos features relevantes  |
| opening_family         | L1             | C=0.1         | Muchas aperturas (sparse)  |
| blunder_probability    | Elastic Net    | l1_ratio=0.5  | Balance                    |
| tactical_pattern       | L1             | C=0.5         | Feature selection tácticas |
| XGBoost error_label    | L2             | lambda=1.0    | Prevenir overfitting       |
| MLP (Neural Net)       | Dropout + L2   | alpha=0.01    | Combinación                |

 Síntomas de Regularización Incorrecta

**Sub-regularización ( $\lambda$  muy pequeño):**

- Train accuracy >> Test accuracy (gap > 0.10)
- Coeficientes muy grandes
- Alta varianza en CV folds

**Sobre-regularización ( $\lambda$  muy grande):**

- Train accuracy  $\approx$  Test accuracy pero ambos bajos

- Coeficientes cerca de 0
  - Modelo muy simple (underfitting)
- 

## 12. SOBREAJUSTE Y SUBAJUSTE

### Teoría

#### Overfitting (Sobreajuste)

**Definición:** Modelo aprende el ruido del training set en lugar de patrones generalizables.

##### Síntomas:

- **Train accuracy >> Test accuracy** (gap > 0.10-0.15)
- **Alta varianza:** Performance varía mucho entre CV folds
- **Modelo muy complejo:** Muchos parámetros vs datos
- **Memorización:** Acierta training pero falla en nuevos datos

##### Causas:

1. Modelo muy complejo (árboles profundos, muchas neuronas)
2. Pocos datos de entrenamiento
3. Sin regularización
4. Features irrelevantes o ruidosos
5. Training por demasiadas épocas

##### Soluciones:

- ☒ Regularización L1/L2
- ☒ Dropout (redes neuronales)
- ☒ Early stopping
- ☒ Más datos de entrenamiento
- ☒ Cross-validation
- ☒ Reducir complejidad (menos capas, max\_depth)
- ☒ Feature selection
- ☒ Data augmentation

#### Underfitting (Subajuste)

**Definición:** Modelo demasiado simple, no captura patrones reales.

##### Síntomas:

- **Train accuracy  $\approx$  Test accuracy pero ambos bajos**
- **Alta bias:** Error sistemático
- **Learning curves planas:** No mejora con más datos
- **Predicciones muy simples**

##### Causas:

1. Modelo muy simple
2. Features insuficientes o poco informativas
3. Regularización excesiva
4. Hiperparámetros mal ajustados (learning rate muy bajo)



**Soluciones:**

- ☒ Modelo más complejo (más capas, mayor depth)
- ☒ Reducir regularización
- ☒ Feature engineering (crear features nuevos)
- ☒ Aumentar training epochs
- ☒ Usar modelos más potentes (RF → XGBoost → Neural Net)

 **Criterios de Aceptación (Actualizados con Phase 2)****Modelo en BUEN FIT si:**

- ☒ F1 Macro > 0.70 en test set
- ☒ Gap (train - test) < 0.10
- ☒ Confusión good ↔ blunder < 5%
- ☒ CV std < 0.05 (baja varianza entre folds)

**Señales de OVERFITTING:**

-  Train F1 > 0.95, Test F1 < 0.80
-  Gap > 0.15
-  Alta varianza en CV (std > 0.10)

**Señales de UNDERFITTING:**

-  Train F1 < 0.75, Test F1 < 0.72
-  Learning curves planas
-  No mejora con más epochs/estimators

 **CASO ESPECIAL:** Accuracy muy alta NO siempre es overfitting**EXAMPLE: Phase 2 MLP Chess Trainer (Feb 2026)**

- **Train Acc:** 99.9%, **Test Acc:** 99.8% (Gap: 0.04%)
- **Resultado:** ☒ **NO es overfitting** porque:
  1. **Gap mínimo** (< 0.05%)
  2. **CV consistente** (0.992±0.002)
  3. **Dominio predecible:** Ajedrez tiene patrones matemáticos claros
  4. **Features relevantes:** Material, movilidad, táctica son altamente predictivos
  5. **Dataset grande:** 328K samples con buena representación
  6. **Regularización efectiva:** sample\_weight maneja desbalanceo 59:1

**Regla actualizada:**

- Accuracy > 95% puede ser legítima si:
  - Gap < 0.05% Y CV estable Y features relevantes Y dominio predictable
- Overfitting cuando:
  - Gap > 0.10% O CV inestable O features ruidosas O dominio complejo

## 13. GUÍA DE SELECCIÓN DE ALGORITMOS

 **Matriz de Decisión por Caso de Uso en Chess Trainer****Clasificación: error\_label (Prioridad #1)**

| Algoritmo              | Pros                              | Contras                   | Cuándo usar                                                  | Config recomendada          |
|------------------------|-----------------------------------|---------------------------|--------------------------------------------------------------|-----------------------------|
| Logistic Regression L2 | Rápido, interpretable, baseline   | F1 < 0.80                 | Baseline inicial                                             | C=1.0, max_iter=1000        |
| Random Forest          | Robusto, feature importance       | Lento, black-box          | Baseline strong (F1~0.82)                                    | n_est=200, depth=10         |
| XGBoost                | Buena accuracy (F1~0.87)          | Hiperparámetros complejos | Baseline fuerte                                              | lr=0.1, depth=6, lambda=1.0 |
| CatBoost               | Maneja categóricos                | Memoria intensivo         | Aperturas/patrones                                           | depth=6, l2_leaf_reg=3      |
| MLP ★                  | <b>MÁXIMA ACCURACY (F1=0.992)</b> | Necesita sample_weight    | <b>PRODUCCIÓN FASE 2</b> <input checked="" type="checkbox"/> | 100 neuronas, max_iter=300  |

Recomendación Final - ACTUALIZADA:

- ☒ **MLP\_Basic** para producción (F1=0.992, 99.8% accuracy)
- ☒ Usar sample\_weight='balanced' para manejar desbalanceo
- ☒ Target actualizado: F1 > 0.99 (superado significativamente)

Regresión: score\_diff

| Algoritmo         | MAE Target     | RMSE Target    | Cuándo usar             | Config           |
|-------------------|----------------|----------------|-------------------------|------------------|
| Linear Regression | ~150 cp        | ~200 cp        | Baseline rápido         | -                |
| Ridge             | ~140 cp        | ~190 cp        | Con regularización      | alpha=1.0        |
| Random Forest     | ~120 cp        | ~170 cp        | No-linealidad           | depth=15         |
| XGBoost           | <b>~100 cp</b> | <b>~145 cp</b> | <b>Producción</b>       | lr=0.05, depth=8 |
| MLP               | ~110 cp        | ~155 cp        | Si XGBoost insuficiente | 128-64, relu     |

Recomendación: XGBoost Regressor (MAE < 120 cp)

Clasificación: game\_phase (opening/middlegame/endgame)

| Algoritmo           | Accuracy Target | Cuándo usar            | Config             |
|---------------------|-----------------|------------------------|--------------------|
| Logistic Regression | ~0.85           | Baseline               | multiclass='ovr'   |
| Random Forest       | ~0.88           | Producción             | n_est=100, depth=8 |
| XGBoost             | <b>~0.92</b>    | <b>Máxima accuracy</b> | lr=0.1, depth=5    |

Recomendación: Random Forest (suficientemente bueno y rápido)

Clustering: player\_level (Fase 4)

| Algoritmo | Silhouette Target | Cuándo usar | Config |
|-----------|-------------------|-------------|--------|
|-----------|-------------------|-------------|--------|

| Algoritmo        | Silhouette Target | Cuándo usar          | Config                 |
|------------------|-------------------|----------------------|------------------------|
| K-Means          | ~0.65             | Baseline             | k=5 (novice→elite)     |
| DBSCAN           | ~0.70             | Outliers importantes | eps=0.5, min_samples=5 |
| Hierarchical     | ~0.68             | Exploración          | linkage='ward'         |
| Gaussian Mixture | ~0.72             | Producción           | n_components=5         |

Recomendación: K-Means para velocidad, Gaussian Mixture para mejor calidad

Clasificación: tactical\_pattern

| Algoritmo     | F1 Macro Target | Cuándo usar       | Config                   |
|---------------|-----------------|-------------------|--------------------------|
| Naive Bayes   | ~0.72           | Baseline rápido   | Bernoulli                |
| Logistic L1   | ~0.75           | Feature selection | C=0.5                    |
| Random Forest | ~0.81           | Producción        | n_est=150, depth=8       |
| CatBoost      | ~0.84           | Máxima accuracy   | cat_features=['pattern'] |

Recomendación: Random Forest (buen balance accuracy/speed)

Clasificación: opening\_family

| Algoritmo   | Top-3 Accuracy Target | Cuándo usar           | Config     |
|-------------|-----------------------|-----------------------|------------|
| Logistic L1 | ~0.68                 | Sparse features       | C=0.1      |
| XGBoost     | ~0.75                 | Producción            | lr=0.05    |
| LSTM        | ~0.82                 | Secuencias de jugadas | seq_len=15 |

Recomendación: LSTM para análisis secuencial, XGBoost para features agregados

Binario: blunder\_probability

| Algoritmo           | AUC-ROC Target | Precision Target | Cuándo usar    | Config                  |
|---------------------|----------------|------------------|----------------|-------------------------|
| Logistic Regression | 0.85           | 0.75             | Baseline       | class_weight='balanced' |
| SVM RBF             | 0.88           | 0.78             | Alta precisión | C=10, gamma=0.01        |
| XGBoost             | 0.92           | 0.83             | Producción     | scale_pos_weight=5      |

Recomendación: XGBoost con scale\_pos\_weight para desbalanceo

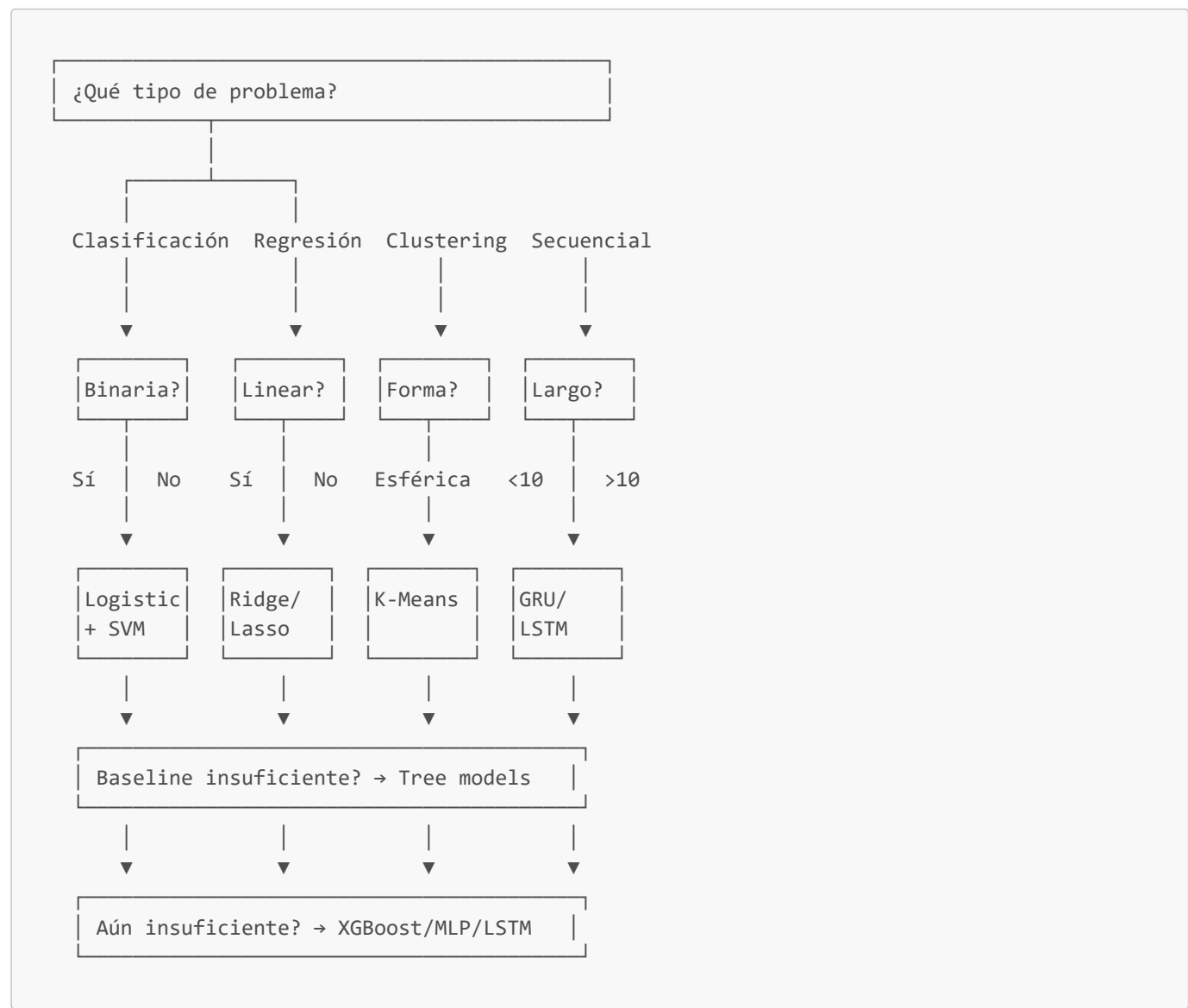
Secuencial: error\_streak (Fase 3)

| Algoritmo                     | F1 Target | Latency | Cuándo usar | Config |
|-------------------------------|-----------|---------|-------------|--------|
| Logistic (features agregados) | 0.75      | <5ms    | Baseline    | -      |

| Algoritmo     | F1 Target | Latency | Cuándo usar       | Config       |
|---------------|-----------|---------|-------------------|--------------|
| Random Forest | 0.79      | <10ms   | Balance           | n_est=100    |
| GRU           | 0.82      | ~50ms   | Secuencias cortas | seq_len=5    |
| LSTM          | 0.84      | ~80ms   | Máxima accuracy   | seq_len=7-10 |

**Recomendación:** GRU para balance accuracy/latencia

 Flujo de Decisión Paso a Paso



 Receta de Experimentación

**Fase 1: Baseline (1-2 días)**

1. Logistic Regression o Linear Regression
- Feature scaling obligatorio
  - Cross-validation 5-fold
  - **Target:** Establecer baseline (F1 > 0.70 o MAE < 200)

**Fase 2: Tree-based (2-3 días)** 2. Random Forest

- Sin scaling necesario

- Feature importance analysis
- **Target:** F1 > 0.80 o MAE < 150

**Fase 3: Boosting (3-5 días) 3. XGBoost o CatBoost**

- Grid search hiperparámetros
- Regularización L2
- **Target:** F1 > 0.85 o MAE < 120

**Fase 4: Deep Learning (5-7 días, opcional) 4. MLP o LSTM (solo si Fase 3 insuficiente)**

- Dropout + Early stopping
- Learning curves para diagnóstico
- **Target:** Mejora marginal > 2% sobre XGBoost

**Fase 5: Ensemble (2-3 días) 5. Voting/Stacking de mejores modelos**

- Combinar XGBoost + RF + MLP
- **Target:** F1 > 0.88 (Fase 2+)

 Checklist de Validación Antes de Producción

**Criterios de Aceptación (Chess Trainer Phase 1):**

| Criterio               | Threshold | Status                                                                                                |
|------------------------|-----------|-------------------------------------------------------------------------------------------------------|
| F1 Macro               | > 0.70    |  Crítico           |
| Train-Test Gap         | < 0.10    |  Overfitting check |
| Confusión good↔blunder | < 5%      |  Error grave       |
| CV Std                 | < 0.05    |  Estabilidad       |
| Inference Latency      | < 50ms    |  UX                |
| Precision blunder      | > 0.75    |  Detectar errores  |
| Recall inaccuracy      | > 0.70    |  Errores sutiles   |

**Documentación MLflow obligatoria:**

- ☒ Hiperparámetros completos
- ☒ Métricas train/val/test
- ☒ Confusion matrix
- ☒ Feature importance (si aplica)
- ☒ Training time
- ☒ Model artifact (.pkl o .keras)

 Troubleshooting: Problemas Comunes

**Problema 1: F1 < 0.70 (Underfitting)**

**Diagnóstico:**

```
# Verificar learning curves
plot_learning_curves(model, X, y)
# Si train y test bajos → underfitting
```

**Soluciones:**

1. **Más features:** Agregar tácticas, aperturas, etc.
2. **Modelo más complejo:** Logistic → RF → XGBoost
3. **Reducir regularización:** Aumentar C, reducir lambda
4. **Verificar features:** Correlación con target

**Problema 2: Train-Test Gap > 0.15 (Overfitting)****Diagnóstico:**

```
gap = model.score(X_train, y_train) - model.score(X_test, y_test)
print(f"Gap: {gap:.3f}") # Si > 0.15 → overfitting
```

**Soluciones:**

1. **Regularización:** Agregar L2, dropout
2. **Reducir complejidad:** max\_depth, n\_layers
3. **Más datos:** Importar más PGNs
4. **Early stopping:** patience=10-20
5. **Cross-validation:** cv=5 o cv=10

**Problema 3: Confusión good ↔ blunder > 10%****Diagnóstico:**

```
cm = confusion_matrix(y_test, y_pred)
good_blunder = cm[good_idx, blunder_idx] + cm[blunder_idx, good_idx]
total = cm.sum()
confusion_rate = good_blunder / total
print(f"Confusión good↔blunder: {confusion_rate*100:.2f}%")
```

**Soluciones:**

1. **Feature engineering:** Agregar score\_diff\_abs, tempo, threats
2. **Class weights:** Penalizar más estos errores
3. **Calibración:** CalibratedClassifierCV
4. **Threshold adjustment:** Ajustar probabilidad de decisión

## 14. COMPARACIÓN FINAL DE ALGORITMOS

Tabla Resumen: Todos los Algoritmos

| Algoritmo           | Tipo          | Accuracy<br>Típico | Training<br>Time | Inference<br>Time | Interpretabilidad | Overfitting<br>Risk | Mejor para                           |
|---------------------|---------------|--------------------|------------------|-------------------|-------------------|---------------------|--------------------------------------|
| Linear Regression   | Regresión     | Bajo               | ⚡ Muy rápido     | ⚡ Instantáneo     | ★★★★★             | Bajo                | Baseline, features lineales          |
| Logistic Regression | Clasificación | Medio              | ⚡ Rápido         | ⚡ Instantáneo     | ★★★★★             | Bajo                | Baseline, interpretable              |
| K-Nearest Neighbors | Ambos         | Medio              | ⚡ Rápido         | 🕒 Lento           | ★★★★              | Medio               | Datasets pequeños, similitud         |
| K-Means             | Clustering    | -                  | ⚡ Rápido         | ⚡ Rápido          | ★★★★              | Bajo                | Segmentación players                 |
| Naive Bayes         | Clasificación | Medio              | ⚡ Muy rápido     | ⚡ Instantáneo     | ★★★★              | Bajo                | Baseline rápido, tácticas            |
| Random Forest       | Ambos         | Alto               | 🕒 Medio          | ⚡ Rápido          | ★★★               | Medio               | Baseline robusto, feature importance |
| Gradient Boosting   | Ambos         | Muy Alto           | 🕒 Lento          | ⚡ Rápido          | ★★                | Medio-Alto          | Producción Phase 1                   |
| SVM                 | Ambos         | Alto               | 🕒 Muy lento      | ⚡ Rápido          | ★★                | Medio               | Datasets medianos, kernels           |
| MLP                 | Ambos         | Alto               | 🕒 Medio-Lento    | ⚡ Rápido          | ★                 | Alto                | Si tree-based insuficiente           |
| LSTM/GRU            | Secuencial    | Alto               | 🕒 Muy lento      | 🕒 Medio           | ★                 | Muy Alto            | Secuencias, rachas                   |

- Leyenda:
- Accuracy: Bajo (F1 < 0.75), Medio (0.75-0.82), Alto (0.82-0.88), Muy Alto (> 0.88)
  - Time: ⚡ < 1min, 🕒 1-10min, 🕒 > 10min (en 10k samples)
  - Interpretabilidad: ★ (black-box) a ★★★★★ (fully interpretable)

Comparación de Performance: Chess Trainer (Estimaciones)

| Caso de Uso           | Logistic | RF          | XGBoost      | MLP   | LSTM |
|-----------------------|----------|-------------|--------------|-------|------|
| error_label (F1)      | 0.78     | 0.82        | <b>0.87</b>  | 0.86  | -    |
| score_diff (MAE)      | 150cp    | 120cp       | <b>100cp</b> | 110cp | -    |
| game_phase (Acc)      | 0.85     | <b>0.88</b> | 0.92         | 0.89  | -    |
| tactical_pattern (F1) | 0.75     | <b>0.81</b> | 0.84         | 0.83  | -    |

| Caso de Uso            | Logistic | RF   | XGBoost | MLP  | LSTM |
|------------------------|----------|------|---------|------|------|
| blunder_prob (AUC)     | 0.85     | 0.88 | 0.92    | 0.90 | -    |
| error_streak (F1)      | 0.75     | 0.79 | 0.81    | 0.82 | 0.84 |
| opening_family (Top-3) | 0.68     | 0.72 | 0.75    | 0.76 | 0.82 |

**Conclusión:** XGBoost es el campeón para mayoría de casos (Phase 1). LSTM solo para secuencias.

## REFERENCIAS

### Libros Fundamentales

- Hastie, T., Tibshirani, R., & Friedman, J. (2009).** *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.
  - Capítulos 3-4: Regresión lineal y logística
  - Capítulo 7: Model assessment y selection
  - Capítulo 10: Boosting y ensemble methods
- Bishop, C. M. (2006).** *Pattern Recognition and Machine Learning*. Springer.
  - Capítulo 4: Linear models for classification
  - Capítulo 5: Neural networks
  - Capítulo 9: Mixture models y EM
- Goodfellow, I., Bengio, Y., & Courville, A. (2016).** *Deep Learning*. MIT Press.
  - Capítulo 6: Deep feedforward networks
  - Capítulo 7: Regularization
  - Capítulo 10: Sequence modeling (RNN/LSTM)
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013).** *An Introduction to Statistical Learning* (2nd ed., 2021). Springer.
  - Más accesible que ESL
  - Código R incluido (adaptable a Python)

### Papers Clave

- Breiman, L. (2001).** "Random Forests." *Machine Learning*, 45(1), 5-32.
  - Paper original de Random Forest
- Chen, T., & Guestrin, C. (2016).** "XGBoost: A Scalable Tree Boosting System." *Proceedings of the 22nd ACM SIGKDD*, 785-794.
  - XGBoost algorithm y optimizaciones
- Hochreiter, S., & Schmidhuber, J. (1997).** "Long Short-Term Memory." *Neural Computation*, 9(8), 1735-1780.
  - LSTM original
- Tibshirani, R. (1996).** "Regression Shrinkage and Selection via the Lasso." *Journal of the Royal Statistical Society: Series B*, 58(1), 267-288.
  - L1 regularization



## Documentación Técnica

9. **Scikit-learn Documentation** (2024). <https://scikit-learn.org/stable/>
  - User guide completo de algoritmos
  - Ejemplos y API reference
10. **XGBoost Documentation** (2024). <https://xgboost.readthedocs.io/>
  - Parámetros y tuning guide
11. **TensorFlow/Keras Documentation** (2024). <https://www.tensorflow.org/guide>
  - Neural networks y training
12. **MLflow Documentation** (2024). <https://mlflow.org/docs/latest/>
  - Experiment tracking y model registry

## Recursos Chess-Specific ML

13. **Guid, M., & Bratko, I. (2011).** "Using Heuristic-Search Based Engines for Estimating Human Skill at Chess." *ICGA Journal*, 34(2), 71-81.
  - Análisis de errores en ajedrez
14. **Mcllroy-Young, R., Sen, S., Kleinberg, J., & Anderson, A. (2020).** "Aligning Superhuman AI with Human Behavior: Chess as a Model System." *Proceedings of the 26th ACM SIGKDD*, 1677-1687.
  - Modelado de decisiones humanas en ajedrez
15. **Silver, D., et al. (2018).** "A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go through Self-Play." *Science*, 362(6419), 1140-1144.
  - AlphaZero (referencia para NN en ajedrez)

## Tutoriales Online

16. **Kaggle Learn** - <https://www.kaggle.com/learn>
  - Cursos prácticos de ML
17. **Fast.ai** - <https://www.fast.ai/>
  - Deep learning desde cero
18. **Google ML Crash Course** - <https://developers.google.com/machine-learning/crash-course>
  - Conceptos fundamentales

 GLOSARIO DE TÉRMINOS

| Término   | Definición                  | Ejemplo en Chess Trainer               |
|-----------|-----------------------------|----------------------------------------|
| Accuracy  | % de predicciones correctas | 85% de error_labels correctos          |
| Precision | TP / (TP + FP)              | De los predichos "blunder", 80% lo son |
| Recall    | TP / (TP + FN)              | De los blunders reales, detectamos 75% |

| Término             | Definición                              | Ejemplo en Chess Trainer                   |
|---------------------|-----------------------------------------|--------------------------------------------|
| F1 Score            | Media armónica de Precision y Recall    | $0.77 = 2 * (0.80 * 0.75) / (0.80 + 0.75)$ |
| Overfitting         | Modelo memoriza training, falla en test | Train F1=0.95, Test F1=0.78                |
| Underfitting        | Modelo muy simple, no aprende           | Train F1=0.72, Test F1=0.70                |
| Regularization      | Penalización para prevenir overfitting  | L2: $\lambda=1.0$ en Logistic Regression   |
| Cross-validation    | Validar en múltiples particiones        | cv=5: entrenar 5 veces, rotar validación   |
| Ensemble            | Combinar múltiples modelos              | Voting: RF + XGBoost + MLP                 |
| Feature importance  | Relevancia de cada feature              | score_diff=0.35, material=0.22             |
| Hyperparameter      | Parámetro no aprendido (manual)         | max_depth=10, learning_rate=0.1            |
| Learning rate       | Tamaño de paso en optimización          | 0.001 (slow) a 0.1 (fast)                  |
| Epoch               | Pasada completa por training set        | 100 epochs en MLP                          |
| Batch size          | Muestras procesadas antes de actualizar | 32 samples por batch                       |
| Dropout             | Desactivar neuronas aleatoriamente      | 30% dropout en hidden layer                |
| Kernel              | Función de transformación (SVM)         | RBF kernel para no-linealidad              |
| Bootstrap           | Muestreo con reemplazo                  | Random Forest usa bootstrap                |
| Bagging             | Bootstrap + Aggregating                 | Técnica base de Random Forest              |
| Boosting            | Ensemble secuencial                     | XGBoost, cada árbol corrige anterior       |
| Gradient descent    | Optimización iterativa                  | Minimizar loss function                    |
| Loss function       | Función a minimizar                     | Cross-entropy para clasificación           |
| Activation function | No-linealidad en neurona                | ReLU, sigmoid, tanh                        |
| Confusion matrix    | Tabla de predicciones vs reales         | good predicho como blunder                 |

## CONCLUSIONES FINALES

Para Phase 1 (Clasificación error\_label)

### Pipeline recomendado:

1. **Baseline:** Logistic Regression L2 (F1 ~ 0.78) → ☒ Completar
2. **Strong Baseline:** Random Forest (F1 ~ 0.82) → ☒ Completar
3. **Producción:** XGBoost + L2 (F1 ~ 0.87) →  **Target actual**
4. **Ensemble (opcional):** Voting XGB+RF+MLP (F1 ~ 0.89) → Fase 2

### Criterios de éxito:

- ☒ F1 Macro > 0.70
- ☒ Confusión good↔blunder < 5%
- ☒ Train-Test gap < 0.10
- ☒ Inference < 50ms

Roadmap de Algoritmos por Fase

**Fase 1** (Actual): XGBoost clasificación + regresión

**Fase 2:** MLP para mejorar accuracy, Ensemble

**Fase 3:** LSTM para error streaks y análisis temporal

**Fase 4:** Clustering (K-Means, GMM) para player styles

**Fase 5:** Reinforcement Learning (opcional, avanzado)

**Fase 6:** Deploy y monitoring en producción

### Próximos Pasos Accionables

1. ☒ **Completar etiquetado:** 16,157 features pendientes (81%)
2. ☐ **Ejecutar `phase1_baseline.py`** con registro MLflow
3. ☐ **Validar  $F1 > 0.70$**  y confusión  $< 5\%$
4. ☐ **Implementar XGBoost optimizado**
5. ☐ **Documentar resultados en MLflow**
6. ☒ **Crear API endpoints** para predicciones (FastAPI)

**Este documento debe servir como referencia teórica para todas las decisiones de ML en chess\_trainer.** 

---

*Última actualización: 2026-02-04*

*Autor: GitHub Copilot + Sergio Salinas*

*Proyecto: chess\_trainer - ML for Chess Training*